



Ministerul Educației
Universitatea "OVIDIUS" Constanța
Facultatea de Matematică și Informatică
Specializarea Informatică

KickSneakApp

Lucrare de licență

Coordonator științific:
Lect. Dr. Andrei Rusu

Absolvent:
Olaru David

Constanța
2026

Cuprins

Cuprins	i
1 Introducere	1
1.1 Motivația alegerii temei	1
1.2 Obiectivele lucrării	2
1.3 Structura lucrării	2
2 Context și stadiul actual	4
2.1 Piața de sale/resale sneakers și apparel	4
2.2 Probleme identificate pe piața existentă	4
2.3 Soluția propusă: KickSneak	6
3 Fundamente teoretice	8
3.1 Arhitectura Clean Architecture	8
3.2 Domain-Driven Design (DDD) — concepte aplicate	10
3.3 Microservicii vs. monolit modular — abordare hibridă	11
3.4 Row Level Security în PostgreSQL	11
3.5 Real-time communication — WebSocket și SignalR	12
3.6 Neural Collaborative Filtering — fundamente matematice	13
3.7 Tehnologii complementare	14

4	Arhitectura sistemului	15
4.1	Vedere de ansamblu	15
4.2	Stack tehnologic	17
4.3	Componente principale și responsabilități	18
4.4	Fluxuri de date end-to-end	18
4.4.1	Scenariul 1: Căutare produs cu sugestii live	18
4.4.2	Scenariul 2: Cumpărare directă cu Stripe	19
4.4.3	Scenariul 3: Licităție cu auto-bid și auto-close	20
4.4.4	Scenariul 4: Chat user-admin (escalare)	21
4.4.5	Scenariul 5: Chat user-LLM (Ollama)	21
4.5	Strategie de deployment și CI/CD	21
4.6	Persistență și Azure Storage	22
5	Securitate aplicată	24
5.1	Filozofia de securitate — Defense in Depth	24
5.2	Pipeline-ul de middleware-uri	25
5.3	SecurityHeadersMiddleware	25
5.4	RateLimiterMiddleware	26
5.5	FirebaseAuthMiddleware	27
5.6	UserResolverMiddleware	28
5.7	TraceMiddleware — Audit și observabilitate	28
5.8	Securitate la nivel de date	29
5.9	Securitate la nivel de cod	29

6	Backend .NET 10	30
6.1	Clean Architecture aplicată	30
6.2	Modelul de date — Entități și relații	30
6.3	RBAC și RLS — implementare Unit of Work	31
6.4	Sistemul de licitații — Redis-first	33
6.5	Integrare Stripe	34
6.6	Minimal APIs și organizarea endpoint-urilor	34
6.7	Background Services — IHostedService și Channels	35
7	Interfața client (React)	37
7.1	Arhitectură și principii de design	37
7.2	Gestiunea stării cu Zustand	38
7.3	Serviciul CachedFetchService	38
7.4	Pagini și fluxuri interactive	38
7.5	Comunicarea în timp real și optimizări	39
8	Microserviciul de chat (Go)	40
8.1	Motivația alegerii Go pentru microserviciul de chat	40
8.2	Arhitectura serviciului	41
8.3	Integrarea Ollama cu streaming	41
8.4	Fluxul de escaladare la operator uman	43
8.5	Persistența sesiunilor și mesajelor în PostgreSQL	43
8.6	Deployment și comunicare cu backend-ul .NET	44

9	Sistem de recomandare AI (Python + PyTorch)	46
9.1	Motivația și alegerea algoritmului	46
9.2	Arhitectura rețelei neuronale	47
9.3	Pipeline-ul de date	47
9.4	Training și evaluare	48
9.5	Servirea modelului via Flask	49
9.6	Integrare cu backend-ul .NET și fallback	49
10	Search, observabilitate și servicii de comunicare	51
10.1	Elasticsearch — indexare și căutare	51
10.2	Kibana	52
10.3	Stack-ul Grafana: Prometheus și Loki	52
10.4	Kotlin — Serviciul de notificări push	53
11	Aplicație administrativă (Next.js + Prisma)	54
11.1	Motivația alegerii Next.js + Prisma	54
11.2	Arhitectura aplicației administrative	55
11.3	Funcționalități administrative principale	55
11.4	Deployment și izolare	56
12	Testare	58
12.1	Strategia de testare	58
12.2	Unit tests — xUnit + NSubstitute + Shouldly	58
12.3	Integration tests — ASP.NET Core TestServer	59
12.4	Automatizare și CI/CD	60

13 Concluzii și direcții viitoare	62
13.1 Rezultate obținute	62
13.2 Limitări și provocări	63
13.3 Direcții viitoare	64
Bibliografie	65

Capitolul 1

Introducere

1.1 Motivația alegerii temei

Dezvoltarea unei platforme moderne de e-commerce de tip *marketplace* ridică probleme tehnice deosebite, a căror rezolvare necesită aplicarea unor modele de proiectare și soluții arhitecturale de ultimă generație. Alegerea acestei teme este fundamentată de dorința de a explora integrarea unor concepte teoretice avansate — precum microserviciile poliglote, securitatea granulară la nivel de bază de date și inteligența artificială aplicată — într-un sistem unitar, robust și scalabil.

Oportunitatea acestei lucrări este susținută de caracterul universal aplicabil al domeniului e-commerce, cea mai mare industrie digitală globală. Logica de implementare dezvoltată pentru un sistem de comerț online (managementul stocurilor, fluxul de achiziție, tranzacțiile securizate și cataloagele de produse) este flexibilă și direct transpozabilă către alte tipuri de platforme software, precum sisteme SaaS, platforme de distribuție de conținut sau piețe virtuale B2B.

Un alt pilon motivațional constă în intensificarea competiției în zona algoritmică de recomandare și căutare a produselor. Lucrarea de față explorează ambele extreme ale acestui spectru tehnologic: căutarea clasică bazată pe relevanță textuală utilizând Elasticsearch [4] și recomandările personalizate realizate prin rețele neuronale profunde de tipul Neural Collaborative Filtering (NCF) antrenate în PyTorch pe seturi de date de interacțiune implicită [9].

În cele din urmă, proiectul abordează provocările arhitecturale reale asociate unui sistem distribuit de producție: securitatea cibernetică (autentificare Firebase [8], RBAC și PostgreSQL RLS [19]), scalabilitatea (decoplare asincronă, modelul *Redis-first* [22])

și observabilitatea (Prometheus, Loki și Grafana). Cuplarea concurenței ridicate la nivel de licitații necesită mecanisme de control optimist și orchestrarea joburilor asincrone prin Quartz.NET [21].

1.2 Obiectivele lucrării

Obiectivul principal al acestei lucrări constă în proiectarea, implementarea și testarea unei platforme complexe de tip marketplace, complet funcționale end-to-end, care integrează un catalog extins, un motor de căutare hibrid, un modul de licitații în timp real, checkout Stripe, un asistent conversațional LLM local, recomandări NCF și un panou administrativ dedicat.

Pentru realizarea acestui scop, au fost stabilite și îndeplinite următoarele obiective secundare:

- ▷ **Aplicarea Clean Architecture și DDD:** Structurarea codului pe straturi decuplate și orientarea pe regulile specifice domeniului de business [5, 11].
- ▷ **Demonstrarea scalabilității prin microservicii poliglote:** Separarea responsabilităților prin implementarea de microservicii în 6 limbaje diferite (.NET, TypeScript, Go, Python, Java și Kotlin).
- ▷ **Implementarea unui sistem real de Machine Learning:** Antrenarea unei rețele neuronale profunde NCF în PyTorch pe feedback implicit [9].
- ▷ **Integrarea unui model de securitate multi-strat:** Corelarea token-urilor Firebase JWT cu reguli RLS și setarea dinamică a rolurilor PostgreSQL [8, 19].
- ▷ **Construirea unui stack de observabilitate:** Monitorizarea stării resurselor și agregarea logurilor și metricilor în Grafana via Prometheus și Loki.
- ▷ **Validarea sistemului prin teste automate:** Implementarea unei strategii de testare unitară și de integrare în backend-ul .NET cu xUnit și NSubstitute.

1.3 Structura lucrării

Lucrarea de față este organizată în 12 capitole care ghidează cititorul prin aspectele teoretice, arhitecturale și practice ale proiectului:

Capitolul 1 — Context și stadiul actual — analizează starea curentă a pieței de comerț electronic și definește soluția KickSneak ca răspuns direct la vulnerabilitățile platformelor existente.

Capitolul 2 — Fundamente teoretice — prezintă Clean Architecture, principiile DDD, topologia microserviciilor poliglote, Row Level Security în PostgreSQL, protocolul WebSocket și bazele matematice NCF.

Capitolul 3 — Arhitectura sistemului — descrie topologia fizică, stack-ul ales și fluxurile end-to-end de date pentru scenariile principale de business.

Capitolul 4 — Securitate aplicată — detaliază middleware-ul de securitate, protecția headerelor, rate limiting-ul, Firebase Auth și aplicarea politicilor RLS.

Capitolul 5 — Backend .NET 10 — descrie structura tehnică a nucleului, designul bazei de date, cache-ul licitațiilor în Redis, checkout-ul Stripe și hosted services.

Capitolul 6 — Interfața client (React) — analizează structura Atomic Design, store-urile Zustand, stilizarea glassmorphism și optimizările UX.

Capitolul 7 — Microserviciul de chat (Go) — descrie serverul WebSocket în Go, integrarea modelului Ollama și clasificatorul de intenții.

Capitolul 8 — Sistem de recomandare AI (Python + PyTorch) — prezintă modelul NCF, antrenarea rețelei și expunerea serviciului prin Flask.

Capitolul 9 — Search, observabilitate și notificări — analizează indexul Elasticsearch, instrumentele Prometheus-Loki-Grafana și microserviciul Kotlin de push notifications.

Capitolul 10 — Aplicație administrativă (Next.js) — detaliază panoul de administrare securizat prin Next.js SSR și Prisma ORM.

Capitolul 11 — Testare — prezintă implementarea testelor unitare și de integrare în backend.

Capitolul 12 — Concluzii și direcții viitoare — sintetizează performanțele realizate și propune modalitățile de migrare în cloud.

Capitolul 2

Context și stadiul actual

2.1 Piața de sale/resale sneakers și apparel

Comerțul electronic (*e-commerce*) s-a consacrat drept structura fundamentală a distribuției de bunuri și servicii. Totuși, presiunea competitivă extremă din acest domeniu forțează o tranziție rapidă de la soluțiile tehnologice tradiționale către sisteme inteligente bazate pe inteligență artificială și procesare distribuită în timp real. Această tranziție se manifestă în mod vizibil în trei direcții principale:

- 1. Evoluția sistemelor de căutare:** Căutările bazate pe interogări SQL clasice direct în bazele de date relaționale au fost înlocuite de motoare de căutare dedicate de tipul Elasticsearch sau Algolia, evoluând spre soluții de căutare semantică (*semantic search*) bazate pe reprezentări vectoriale (*embeddings*).
- 2. Personalizarea recomandărilor:** Recomandările simple bazate pe popularitatea generală au făcut loc rețelelor neuronale profunde. Modelele contemporane, precum filtrarea colaborativă neuronală (*Neural Collaborative Filtering — NCF*) [9], analizează feedback-ul implicit pentru a prezice afinitatea utilizatorilor.
- 3. Asistența conversațională avansată:** Chatbot-urile bazate pe reguli rigide sunt înlocuite rapid de asistenți conversaționali integrați cu modele mari de limbaj (*LLMs*), capabili să interpreteze limbajul natural în timp real și să păstreze contextul discuției.

2.2 Probleme identificate pe piața existentă

În ciuda avansului tehnologic descris, o analiză detaliată a multor marketplace-uri și platforme de e-commerce existente relevă o serie de deficiențe tehnice recurente, care afec-

tează negativ atât securitatea utilizatorilor, cât și calitatea interacțiunii acestora cu sistemul.

O problemă frecventă este utilizarea căutării clasice direct în bazele de date relaționale prin interogări SQL rigide de tip ‘LIKE ’%...%’’. Acestea sunt extrem de ineficiente pe volume mari de date, ducând la scanări complete de tabele și blocarea conexiunilor la baza de date, fără a oferi toleranță la greșeli de tipar (*typo tolerance*) sau potriviri aproximative (*fuzzy matching*).

Absența unor modele reale de personalizare bazate pe învățarea automată (*Machine Learning*) reprezintă o altă deficiență majoră. Majoritatea platformelor folosesc algoritmi bazați strict pe popularitatea globală a produselor, ceea ce duce la recomandări irelevante contextual din cauza lipsei reprezentărilor vectoriale ale utilizatorilor și produselor.

Multe platforme comerciale promovează chatbot-uri rigide bazate pe arbori de decizie scriptați și potriviri de cuvinte cheie. Utilizatorul este constrâns să folosească formule lingvistice predefinite exacte pentru a obține o informație simplă, în lipsa unei înțelegeri semantice, a memoriei conversaționale sau a abilității de a genera răspunsuri fluide.

În special pe platformele de resale care folosesc licitații sau oferte, transparența datelor este adesea deficitară. Utilizatorii nu au acces în timp real la un istoric complet al ofertelor depuse (*bid history*), la evoluția prețurilor sau la actualizări privind statusul comenzilor lor, depinzând de refresh-uri manuale.

O vulnerabilitate de securitate gravă o reprezintă stocarea locală a identității utilizatorilor în baza de date proprie, folosind adesea algoritmi de hashing depășiți (cum ar fi MD5 sau SHA1). Aceste sisteme nu oferă protecție împotriva atacurilor de tip brute-force, nu integrează autentificarea cu doi factori (2FA) și evită delegarea către provideri specializați precum Firebase Authentication [8].

Multe implementări legacy folosesc procese de plată custom care încalcă normele PCI-DSS [17], colectând sau salvând datele sensibile ale cardurilor direct în baza de date locală a aplicației pentru a efectua procesarea de plată, în loc să utilizeze tokenizarea oferită de procesoare certificate precum Stripe [24].

La nivel de infrastructură internă, platformele suferă din cauza structurilor monolitice rigide și a lipsei totale de observabilitate. Modificarea unei funcționalități mici necesită redeploy-ul întregului sistem, iar depanarea erorilor în producție este îngreunată de absența log-urilor centralizate, a metricilor în timp real și a tracing-ului distribuit.

2.3 Soluția propusă: KickSneak

Aplicația KickSneak a fost proiectată ca un răspuns direct și coerent la fiecare dintre problemele identificate mai sus, integrând bunele practici tehnice recunoscute la nivel industrial într-o arhitectură modulară distribuită.

- 1. Căutare rapidă și flexibilă:** Căutarea clasică SQL a fost înlocuită cu un motor Elasticsearch [4], sincronizat asincron din PostgreSQL printr-un pipeline bazat pe canale in-memory în .NET ('Channel<T>').
- 2. Interacțiune sub 100ms:** Căutarea live din frontend folosește o conexiune SignalR directă cu backend-ul, atingând un timp total de răspuns sub 100ms prin debouncing client-side și interogarea indexului Elasticsearch.
- 3. Recomandări personalizate bazate pe AI:** Integrarea unei rețele neuronale custom Neural Collaborative Filtering (NCF) în PyTorch [9] antrenată pe feedback implicit, oferind sugestii relevante contextual.
- 4. Asistent conversațional LLM local:** Serviciul de chat utilizează local modelul *llama3.1:8b* prin Ollama [16], rulat pe conexiuni WebSocket în Go, și include un clasificator pentru escaladare la operator uman.
- 5. Transparență totală în timp real:** Transmiterea instantanee a update-urilor de bidduri și a statusurilor comenzilor către clienți prin conexiuni persistente SignalR.
- 6. Autentificare securizată federată:** Delegarea identității către Firebase Authentication [8], cu validare criptografică a token-urilor JWT la fiecare request și monitorizarea IP în Redis pentru detecția anomaliilor.
- 7. Plăți securizate prin Stripe:** Utilizarea Stripe Checkout [24] pentru a preveni accesul serverelor KickSneak la datele cardurilor, tranzacțiile fiind confirmate asincron prin webhook-uri semnate criptografic.
- 8. Observabilitate profesională:** Agregarea metricilor Prometheus și a log-urilor Loki în dashboard-uri Grafana utilizând identificatori de trace ('TraceId') corelați.
- 9. Arhitectură hibridă decuplată:** Nucleul tranzacțional implementat ca un monolit modular în .NET 10 (Clean Architecture), iar serviciile auxiliare rulate ca microservicii independente prin Docker Compose [3].

Sistemul KickSneak se diferențiază prin integrarea a 6 limbaje de programare (C#, TypeScript, Go, Python, Java și Kotlin), utilizarea unui model hibrid de securitate în baza de date (RBAC + RLS activat în PostgreSQL [19]) forțat din backend și administrarea licitațiilor printr-un mecanism rapid de tip *Redis-first* cu control optimist al concurenței [22]. Ca limitări

asumate academic, sistemul rulează momentan local prin Docker Compose, iar setul de date de antrenare PyTorch este generat semi-sintetic local, constituind direcții viitoare de dezvoltare.

Capitolul 3

Fundamente teoretice

3.1 Arhitectura Clean Architecture

Arhitectura software reprezintă fundamentul pe care se clădește fiabilitatea, menținabilitatea și evoluția în timp a oricărui sistem complex. În cadrul proiectării nucleului platformei KickSneak, s-a optat pentru utilizarea modelului de design *Clean Architecture*, conceptualizat de Robert C. Martin (cunoscut în comunitate sub prezumția *Uncle Bob*) în lucrarea sa fundamentală [11].

Principiul de bază al acestui tip de arhitectură este regula dependenței (*Dependency Rule*), care stipulează că toate dependențele de cod trebuie să fie orientate exclusiv dinspre straturile exterioare (detaliile tehnice, cum ar fi bazele de date, framework-urile de UI sau integrările terțe) spre straturile interioare (conceptele de business, entitățile și cazurile de utilizare). Straturile interne nu cunosc nicio informație despre modul în care datele sunt persistate sau despre protocoalele de comunicare.

În implementarea realizată în platforma KickSneak folosind framework-ul .NET 10, structura a fost organizată în următoarele cinci straturi concentrice:

- 1. Domain (Domeniu):** Reprezintă nucleul sistemului și conține entități pure (*Entities* cum ar fi *Product*, *Auction*, *Order*), obiecte valoare (*Value Objects* de tipul *Address*), enumerări (*DbRole*) și excepții specifice domeniului, fiind scris în cod C# pur.
- 2. Application (Aplicație):** Conține logica de business specifică aplicației și cazurile de utilizare, definind serviciile (*AuctionService*, *ProductService*), obiectele de transfer de date (*DTOs*) și interfețele de abstractizare (*IUnitOfWork*, *IRepository*).
- 3. Persistence (Persistență):** Reprezintă implementarea concretă a persistenței datelor utilizând ORM-ul Entity Framework Core (EF Core), contextul bazei de date

(ApplicationDbContext), configurările Fluent API și repozitoriile concrete.

4. **Infrastructure (Infrastructură):** Gestionează conectivitatea cu toate serviciile externe și bibliotecile third-party, precum checkout-ul Stripe, clientul Elasticsearch, conexiunile Redis, Azure Blob Storage și Firebase Admin SDK.
5. **WebApi (Prezentare / API):** Stratul cel mai exterior, care expune funcționalitățile sub formă de Minimal APIs în .NET 10, middleware-uri, Hub-uri SignalR și configurările de Dependency Injection (DI).

Separarea strictă a straturilor oferă avantaje practice imediate. Decuplarea logicii de business de baza de date permite înlocuirea facilă a motorului de stocare (de exemplu, migrarea de la PostgreSQL la SQL Server) fără a modifica vreo linie de cod din stratul de aplicație. De asemenea, se previne apariția dependențelor circulare, deoarece compilatorul C# va genera erori de compilare dacă o entitate din Domain ar încerca să referențieze un serviciu din Application.

Arhitectura respectă principiile SOLID [11]: Single Responsibility Principle (SRP) prin delegarea fiecărui caz de utilizare unui handler dedicat (ex. PlaceBidHandler); Open/Closed Principle (OCP) prin extinderea funcționalităților prin noi servicii fără modificarea codului existent; Liskov Substitution Principle (LSP) prin substituirea repozitoriilor concrete cu abstracțiile lor; Interface Segregation Principle (ISP) prin utilizarea de interfețe mici și dedicate (ex. IAuctionRepository); și Dependency Inversion Principle (DIP) prin decuplarea modulelor de nivel înalt (Application) de cele de nivel inferior (Persistence/Infrastructure) prin intermediul abstracțiilor.

O altă componentă critică este centralizarea tranzacțiilor printr-un model de tip Unit of Work (IUnitOfWork), care expune repozitoriile și se asigură că toate operațiunile de scriere din cadrul unui request sunt salvate într-o singură tranzacție atomică la nivel de bază de date. De asemenea, utilizarea interfețelor permite scrierea facilă de teste unitare, dependențele putând fi simulate (*mock-uite*) cu biblioteci precum NSubstitute fără a necesita o bază de date reală sau servicii active în fundal. Din perspectivă teoretică, Clean Architecture reprezintă o evoluție și o sinteză a modelelor anterioare de organizare software, precum arhitectura stratificată clasică, *Hexagonal Architecture* și *Onion Architecture*.

3.2 Domain-Driven Design (DDD) — concepte aplicate

Proiectarea orientată pe domeniu (*Domain-Driven Design* — *DDD*), introdusă de Eric Evans în lucrarea sa clasică [5], este o paradigmă software utilizată pentru a gestiona complexitatea sistemelor prin maparea strânsă a structurii codului la modelul de business real. În KickSneak, s-au extras și aplicat mai multe elemente fundamentale ale acestei teorii.

Limbajul universal (*Ubiquitous Language*) este aplicat cu rigurozitate: termenii utilizați în baza de date și în codul sursă reflectă fidel jargonul din industria de resale. Concepte precum Auction, Bid, Listing, StockItem și Seller înlocuiesc denumirile generice de baze de date.

La nivelul modelării datelor, se face o distincție clară între entități (*Entities*) și obiecte valoare (*Value Objects*). Entitățile, cum ar fi clasa Product, sunt definite printr-o identitate unică care persistă pe tot parcursul ciclului de viață. Obiectele valoare, cum ar fi Address, sunt imutabile și se definesc exclusiv prin valorile atributelor lor, neavând nevoie de un ID unic în bază.

Sistemul utilizează conceptul de agregate (*Aggregates*) și rădăcini de agregat (*Aggregate Roots*). Rădăcina de agregat garantează consistența tranzacțională a entităților asociate. De exemplu, clasa Auction este o rădăcină de agregat care încapsulează o listă de obiecte Bid. Orice adăugare sau modificare a unui bid nu se poate face direct pe tabela de bid-uri, ci trebuie să treacă prin metodele interne ale agregatului Auction (ex. `auction.PlaceBid(userId, amount)`), care validează regulile de business.

Pentru decuplarea asincronă la nivel de proces în nucleul .NET, se utilizează primitivele de concurență `Channel<T>`. La modificarea unui produs, se publică un eveniment într-un `Channel<ElasticSyncEvent>` procesat asincron de un worker de background, iar evenimentele de sistem sunt scrise într-un `Channel<NotificationEvent>` și distribuite prin SignalR. Folosirea `Channel<T>` in-process elimină overhead-ul de rețea în stadiul curent de dezvoltare, lăsând arhitectura pregătită pentru integrarea ulterioară a unui broker extern (ex. RabbitMQ).

3.3 Microservicii vs. monolit modular — abordare hibridă

Modelul de tip **monolit modular** presupune că toată logica rulează într-un singur proces divizat în module strict izolate, oferind simplitate de deploy și tranzacții ACID garantate nativ, având însă dezavantajul scalării uniforme obligatorii și a rigidității tehnologice. În schimb, arhitectura de **microservicii** rulează procese independente în containere separate, permițând scalabilitate granulară și izolare la erori, cu prețul unei complexități operaționale sporite și a necesității de a gestiona consistența eventuală.

KickSneak propune o ****abordare hibridă pragmatică**** (*Pragmatic Distributed Monolith*): nucleul tranzacțional al business-ului (produse, licitații, comenzi, checkout) este implementat ca un monolit modular în .NET 10, asigurând consistența ACID. În paralel, serviciile cu cerințe speciale rulează independent: microserviciul de chat (scris în Go, optim pentru WebSockets concurente cu goroutine-uri consumând doar 2KB de memorie fiecare), modulul de recomandări AI (Python cu PyTorch pentru optimizările tensorilor) și panoul administrativ (Next.js/Prisma pentru securitate și port izolat). Comunicarea dintre servicii se bazează pe API-uri HTTP REST pentru interogări simple și conexiuni WebSocket/SignalR pentru fluxurile de date în timp real.

3.4 Row Level Security în PostgreSQL

Securitatea la nivel de rând (*Row Level Security — RLS*), introdusă nativ începând cu versiunea PostgreSQL 9.5 [19], reprezintă un mecanism avansat de control al accesului la date. Spre deosebire de securitatea clasică aplicată în stratul de aplicație (unde un programator poate uita din greșeală să adauge clauza `WHERE user_id = current_user` într-o interogare), RLS impune politicile de filtrare direct în motorul bazei de date. Niciun query nu poate returna sau modifica înregistrări la care rolul conectat nu are drept de acces.

Modelul de date al platformei KickSneak implementează un sistem cu 6 roluri PostgreSQL distincte:

1. `ks_guest`: Rol utilizat pentru vizitatorii neautentificați; are drepturi stricte de lectură (SELECT) doar pe tabelele din catalogul public de produse.

2. `ks_user`: Rol atribuit cumpărătorilor autentificați; are drepturi de scriere și citire pe propriile comenzi, favorite și coș de cumpărături.
3. `ks_seller`: Rol destinat vânzătorilor; moștenește permisiunile de `ks_user` și adaugă acces la modificarea propriilor listări de produse și vizualizarea tranzacțiilor aferente.
4. `ks_chat_service`: Rol securizat alocat exclusiv microserviciului de chat în Go, limitat strict la tabelele de mesagerie și sesiuni de chat.
5. `ks_admin`: Rol cu acces total pe toate tabelele bazei de date, utilizat de aplicația administrativă din Next.js.
6. `ks_owner`: Rolul principal utilizat la runtime de conexiunea backend-ului .NET; acesta deține drepturile de creare tabele și are permisiunea de a schimba identitatea conexiunii curente prin instrucțiuni `SET ROLE`.

Politicile RLS sunt activate explicit pe tabelele sensibile prin clauza `ALTER TABLE ... ENABLE ROW LEVEL SECURITY`. Pentru a stabili identitatea utilizatorului din aplicație în PostgreSQL, backend-ul execută în cadrul fiecărei tranzacții o instrucțiune SQL preliminară utilizând micro-ORM-ul Dapper:

```
1 SET LOCAL ROLE ks_user;  
2 SELECT set_config('app.current_user_id', @UserId, true);
```

Comanda `SET LOCAL` garantează că rolul stabilit este valabil strict pe durata tranzacției curente, prevenind astfel atacurile de tip "scurgere de rol" (*role leakage*) între request-uri concurente. Politicile RLS din PostgreSQL sunt configurate să citească această setare locală prin funcția `current_setting('app.current_user_id')`, realizând filtrarea dinamică a înregistrărilor.

3.5 Real-time communication — WebSocket și SignalR

Protocoalele HTTP tradiționale funcționează pe modelul cerere-răspuns (*request-response*), în care clientul inițiază tranzacția, iar serverul răspunde și închide conexiunea. Pentru funcționalități interactive precum licitațiile live sau chat-ul conversational, acest model este inefficient, necesitând tehnici costisitoare de *polling*.

Protocolul WebSocket, definit în specificația RFC 6455, stabilește o conexiune bidirecțională, persistentă și full-duplex peste un singur canal TCP. Odată finalizată strângerea

de mână inițială (*handshake-ul*) prin HTTP, comunicarea se realizează prin cadre de date cu overhead minim de rețea.

În platforma KickSneak, comunicarea în timp real este structurată pe două tehnologii:

- ▷ **SignalR (.NET)**: Reprezintă o abstractizare de nivel înalt peste protocolul WebSocket, oferind fallback-uri automate la Server-Sent Events (SSE) sau Long Polling în cazul în care clientul sau rețeaua nu suportă WebSockets. SignalR facilitează conceptul de grupuri, permițând gruparea utilizatorilor pe camere specifice (ex. grupul `auction-{id}`). Când un utilizator plasează un bid, serverul transmite noul preț direct grupului asociat licitației respective.
- ▷ **WebSocket Nativ (Go)**: Utilizat în microserviciul de chat. S-a optat pentru WebSocket nativ datorită flexibilității maxime oferite în transmiterea răspunsurilor în flux (*streaming*) de la modelul de limbaj Ollama, permițând afișarea textului caracter cu caracter pe client.

3.6 Neural Collaborative Filtering — fundamente matematice

Sistemul de recomandare AI din KickSneak se bază pe algoritmul *Neural Collaborative Filtering* (NCF) [9], care înlocuiește factorizarea matricială clasică cu o structură neuronală profundă capabilă să capteze interacțiuni non-liniare. Pentru utilizatorul $u \in U$ și produsul $i \in I$, sistemul învață reprezentări vectoriale dense (embeddings) $e_u, e_i \in \mathbb{R}^d$ (unde $d = 32$) extrase prin:

$$e_u = P^T v_u^U \in \mathbb{R}^d \quad (3.1)$$

$$e_i = Q^T v_i^I \in \mathbb{R}^d \quad (3.2)$$

unde P și Q sunt matricile de embeddings ale utilizatorilor și produselor, iar v_u^U și v_i^I sunt vectorii one-hot ai ID-urilor de intrare. Stratul Generalized Matrix Factorization (GMF) modelează interacțiunile liniare prin produsul element cu element (dot product):

$$\phi^{GMF} = e_u \odot e_i \quad (3.3)$$

În paralel, Multi-Layer Perceptron (MLP) captează asocierile non-liniare prin concatenarea vectorilor de embedding și propagarea lor prin straturi liniare activate cu funcția ReLU:

$$\phi_0^{MLP} = [e_u; e_i] \quad (3.4)$$

$$\phi_1^{MLP} = a_1(W_1^T \phi_0^{MLP} + b_1) \quad (3.5)$$

$$\phi_L^{MLP} = a_L(W_L^T \phi_{L-1}^{MLP} + b_L) \quad (3.6)$$

unde W_k , b_k și a_k reprezintă ponderile, bias-ul și funcția de activare pentru stratul k (structurat ca $64 \rightarrow 32 \rightarrow 16$). Ieșirile din straturile GMF și MLP sunt combinate și trecute printr-o funcție de activare *Sigmoid* pentru a calcula probabilitatea ca utilizatorul u să interacționeze favorabil cu produsul i :

$$\hat{y}_{ui} = \sigma(h^T[\phi^{GMF}; \phi^{MLP}]) \quad (3.7)$$

unde $\sigma(x) = \frac{1}{1+e^{-x}}$ este funcția sigmoidă standard, iar h este vectorul de predicție final. Deoarece sistemul utilizează feedback implicit, ponderile sunt optimizate prin algoritmul Adam minimizând funcția de pierdere Binary Cross-Entropy (BCE) pe setul de interacțiuni pozitive Y și negative Y^- :

$$L = - \sum_{(u,i) \in Y \cup Y^-} (y_{ui} \log \hat{y}_{ui} + (1 - y_{ui}) \log (1 - \hat{y}_{ui})) \quad (3.8)$$

3.7 Tehnologii complementare

Pentru a facilita implementarea modelelor descrise, s-au utilizat o serie de tehnologii suplimentare suport. Elasticsearch acționează ca bază de date de căutare secundară, organizând produsele într-un index inversat bazat pe relevanță TF-IDF / BM25. Redis stochează stările licitațiilor în memorie sub formă de tabele hash pentru acces ultrarapid, oferind suport pentru tranzacții atomice prin comenzi de urmărire WATCH. Prometheus și Loki colectează datele de telemetrie, Prometheus preluând metricile expuse de serviciile .NET, iar Loki agregând logurile Docker în timp real, ambele fiind conectate ca surse de date în Grafana pentru vizualizare.

Capitolul 4

Arhitectura sistemului

4.1 Vedere de ansamblu

Aplicația KickSneak adoptă o arhitectură distribuită hibridă. Aceasta combină stabilitatea și consistența tranzacțională a unui nucleu .NET 10 cu performanțele specifice ale unor microservicii în Go și Python, deserving două interfețe frontend independente realizate în React și Next.js. Sistemul este structurat pe patru niveluri funcționale principale:

1. **Layer-ul de Prezentare (Presentation Layer):** Storefront-ul client (React 18, TypeScript, Vite) pentru utilizatori și Admin Panel-ul (Next.js, TypeScript, Prisma) pentru moderare și management de stoc.
2. **Layer-ul API (Core Backend):** Scris în .NET 10, validează securitatea prin Firebase SDK și acționează ca un API gateway intern pentru microservicii.
3. **Layer-ul de Servicii (Service Layer):** Microserviciul de chat în Go (WebSockets) și microserviciul AI în Python (Flask/PyTorch) pentru recomandări.
4. **Layer-ul de Date și Infrastructură (Data Layer):** Centralizează persistența în PostgreSQL 16, cache-ul licitațiilor în Redis 7, căutarea în Elasticsearch 8 și stocarea de imagini în Azure Storage.

Figura 4.1 ilustrează modul în care componentele KickSneak comunică între ele și partajează resursele de persistență.

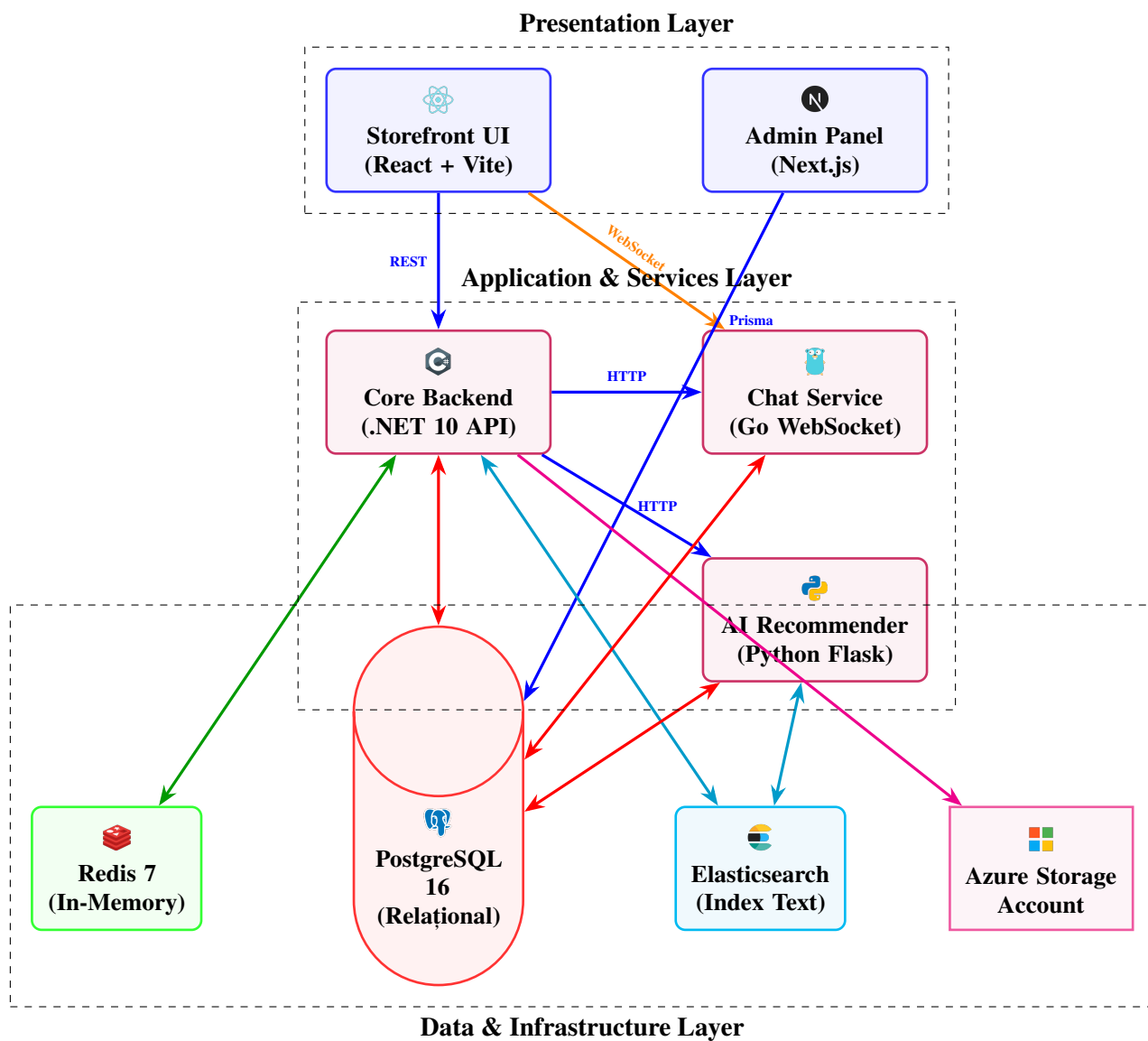


Figura 4.1: Diagrama de arhitectură de nivel container (C4) pentru sistemul KickSneak

4.2 Stack tehnologic

Fiecare componentă a sistemului KickSneak a fost selectată pe baza unui proces de evaluare tehnică riguros, asigurând un echilibru optim între performanță, compatibilitate și fiabilitate.

- ▷ **React 18, Vite și TypeScript** (Storefront UI): React oferă un model declarativ stabil bazat pe componente pentru interfață. Vite garantează timpi de încărcare ultra-rapizi la dezvoltare prin utilizarea modulelor ES native, iar TypeScript asigură siguranța tipurilor la compilare, prevenind erorile runtime.
- ▷ **Next.js 14 și Prisma** (Admin Panel): Next.js utilizează randarea pe server (SSR) pentru a spori securitatea în panoul administrativ, limitând expunerea codului sensibil pe client. Prisma servește ca ORM, generând automat tipuri statice direct din schema bazei de date.
- ▷ **.NET 10, EF Core și Dapper** (Backend Core): .NET 10 asigură un runtime de înaltă performanță pentru Minimal APIs. EF Core este ORM-ul principal pentru logica complexă, în timp ce micro-ORM-ul Dapper este folosit pentru performanță brută în execuția interogărilor SQL directe și a tranzacțiilor rapide.
- ▷ **PostgreSQL 16**: Bază de date relațională utilizată ca sursă principală de adevăr tranzacțional. Suportă structuri JSON pentru metadate dinamice și implementează securitatea la nivel de rând (Row Level Security) direct din motorul bazei de date.
- ▷ **Go** (Chat Service): Limbaj compilat cu performanțe remarcabile în procesarea concurențială. Este ideal pentru conexiuni WebSocket persistente datorită modelului CSP bazat pe goroutine-uri ușoare și canale de comunicare rapide.
- ▷ **Python, Flask și PyTorch** (AI Service): Python oferă ecosistemul optim pentru învățarea automată. PyTorch este utilizat pentru a rula modelul NCF de recomandări, iar micro-framework-ul Flask expune logica de inferență prin API-uri REST.
- ▷ **Elasticsearch 8**: Asigură motorul de căutare full-text secundar, indexând catalogul de produse pentru a permite căutări aproximative rapide bazate pe relevanță textuală.
- ▷ **Redis 7**: Bază de date in-memory utilizată pentru cache și pentru stocarea stării volatile a licitațiilor active, garantând tranzacții atomice cu latențe sub o milisecundă.
- ▷ **Ollama**: Rulează local modele mari de limbaj (cum ar fi *llama3.1:8b*), garantând confidențialitatea totală a conversațiilor utilizatorilor și eliminând dependența de API-uri terțe.
- ▷ **Azurite**: Emulator local pentru Azure Storage. Expune API-urile pentru Azure Blob (stocare imagini produse) și Table Storage (jurnale de audit), asigurând compatibilitate

completă cu deployment-urile cloud.

4.3 Componente principale și responsabilități

Componentă	Rol primar	Tehnologie	Comunică cu (Protocoale)
Storefront FE	Interfață publică	React, Vite, TS	Backend Core (HTTP/SignalR), Chat (WebSocket)
Admin FE	UI administrare	Next.js, Prisma	PostgreSQL direct (rol ks_admin)
Backend Core	Core Business Logic	.NET 10, EF Core	Postgres, Redis, ES, Firebase, Stripe, Azure Storage Account
Chat Service	Asistent conversational	Go, WebSocket	PostgreSQL (rol ks_chat_service), Ollama
AI Recommender	Recomandări neuronale	Python, PyTorch	PostgreSQL (readonly), Elasticsearch
PostgreSQL 16	Bază de date	Postgres 16	Backend Core, Chat, Admin FE
Redis 7	Cache / Auctions	Redis 7	Backend Core
Elasticsearch 8	Index căutare	ES 8	Backend Core, AI Recommender
Azure Storage Account	Stocare Blob & Tabele	Azurite SDK	Backend Core
Ollama	Motor LLM local	Ollama SDK	Chat Service (HTTP)
Prometheus	Colectare metrice	Prometheus	Toate serviciile (Scrape HTTP)
Loki	Agregare loguri	Loki / Promtail	Docker daemon
Grafana	Vizualizare telemetrie	Grafana	Prometheus, Loki

Tabela 4.1: Matricea componentelor sistemului KickSneak și protocoalele de comunicare

4.4 Fluxuri de date end-to-end

Pentru a înțelege cum interacționează componentele, sunt analizate în continuare cinci scenarii critice de rulare a aplicației.

4.4.1 Scenariul 1: Căutare produs cu sugestii live

1. Utilizatorul introduce caractere în caseta de căutare din frontend.
2. Pe client se aplică un mecanism de *debouncing* de 300ms. Dacă utilizatorul continuă să scrie, nu se trimite niciun request.

3. Când scrierea se oprește, clientul apelează prin SignalR metoda `SearchHub.SearchAsync(query)`.
4. Backend-ul Core interoghează indexul Elasticsearch prin interogări de tip `multi_match` cu parametrul `fuzziness` setat pe `AUTO` (pentru a tolera greșelile de tipar).
5. Elasticsearch returnează documentele potrivite sortate după scorul BM25.
6. Dacă utilizatorul are activat filtrul "Search with AI", backend-ul apelează suplimentar microserviciul Python la endpoint-ul `POST /api/recommend`, obținând o listă de ID-uri ordonate conform vectorului de preferințe al utilizatorului.
7. Backend-ul re-mapează datele și le trimite înapoi prin conexiunea activă SignalR, rezultatele fiind afișate sub forma unui meniu dropdown instantaneu.

4.4.2 Scenariul 2: Cumpărare directă cu Stripe

1. Utilizatorul adaugă un produs în coș (stare sincronizată local și salvată în baza de date PostgreSQL prin backend).
2. În pagina de checkout, utilizatorul introduce datele de livrare și alege plata prin card bancar, trimițând un request la backend-ul .NET pe endpoint-ul `POST /api/checkout/create-session`.
3. Backend-ul validează stocurile și apelează API-ul Stripe pentru a genera o sesiune securizată de plată (`checkout.sessions.create`), trimițând metadata asociată comenzii (ID-ul comenzii interne și ID-ul utilizatorului).
4. Stripe returnează URL-ul sesiunii de checkout. Backend-ul trimite acest URL clientului, care redirecționează utilizatorul pe pagina securizată Stripe.
5. Utilizatorul introduce datele cardului pe pagina Stripe, beneficiind de autentificarea 3D Secure dacă este necesar.
6. Stripe confirmă plata și redirecționează utilizatorul pe o pagină de succes din frontend.
7. Asincron, serverele Stripe trimit un eveniment semnat criptografic de tip `checkout.session.completed` pe endpoint-ul de webhook al backend-ului `/webhooks/stripe`.
8. Backend-ul validează semnătura webhook-ului, marchează comanda ca plătită (Paid) și trimite asincron un eveniment în `Channel<NotificationEvent>` pentru a notifica utilizatorul live prin SignalR și a genera factura în fundal.

4.4.3 Scenariul 3: Licitatie cu auto-bid și auto-close

1. Vânzătorul creează o licitație. Backend-ul Core o persistă în PostgreSQL și, simultan, inițializează starea licitației în baza de date Redis sub cheia `auction:{id}` ca un hash ce conține oferta curentă, timpul de finalizare și ID-ul ultimului ofertant.
2. Backend-ul înregistrează un job asincron în Quartz.NET programat să ruleze la data de expirare a licitației (`AuctionCloseJob`).
3. Când un cumpărător vizualizează pagina detaliată, clientul React se conectează la SignalR Hub-ul `AuctionsHub` și se alătură grupului `auction-{id}`.
4. Cumpărătorul plasează o ofertă. Request-ul ajunge la backend, care utilizează tranzacțiile Redis de tip optimistic locking (`WATCH / MULTI / EXEC`) pentru a asigura consistența concurențială:

```
1      // Cod conceptual de plasare bid in Redis
2      db.Watch(auctionKey);
3      var currentBid = db.HashGet(auctionKey, "currentBid");
4      if (newBid > currentBid) {
5          var transaction = db.CreateTransaction();
6          transaction.HashSetAsync(auctionKey, "currentBid", newBid
7      );
8          transaction.HashSetAsync(auctionKey, "currentBidder",
9      userId);
10         bool success = transaction.Execute();
11         // Daca success e false, tranzactia a esuat din cauza
12         unui bid concurent
13     }
```

5. Dacă pe licitație există setat un prag maxim de auto-bid de către un alt utilizator, sistemul verifică dacă noul bid este mai mic decât pragul respectiv și plasează automat o contra-ofertă în mod recursiv până când se stabilește un nou câștigător.
6. Backend-ul notifică grupul SignalR `auction-{id}` cu noul status, actualizând ecranele tuturor utilizatorilor conectați în timp real.
7. La data finalizării, Quartz.NET execută `AuctionCloseJob`, scrie starea finală din Redis în PostgreSQL, generează comanda automat pentru câștigător și închide licitația.

4.4.4 Scenariul 4: Chat user-admin (escalare)

1. Clientul React inițiază o conexiune WebSocket cu microserviciul Go la `/ws/user-chat`, trimițând token-ul JWT Firebase în query string.
2. Serviciul în Go validează token-ul JWT și înregistrează sesiunea cu statusul activ în tabela `chat_sessions` din PostgreSQL.
3. Mesajele trimise de utilizator sunt persistate în baza de date și retransmise instant operatorilor conectați în Admin Panel Next.js prin WebSocket-ul dedicat (`/ws/admin-chat`), iar răspunsurile acestora urmează calea inversă.

Detaliile tehnice complete ale microserviciului de chat sunt prezentate în Capitolul 8.

4.4.5 Scenariul 5: Chat user-LLM (Ollama)

1. Utilizatorul deschide o conexiune WebSocket la Go Chat Service pe calea `/ws/ai-chat`.
2. La primirea unui mesaj, Go Chat Service analizează intenția: dacă se solicită asistență umană, sesiunea este marcată cu `escalated=true` și redirecționată către operator.
3. În caz contrar, se interoghează local API-ul Ollama (*llama3.1:8b*), iar token-urile generate sunt trimise în timp real (*streaming*) către client și apoi persistate.

Detaliile tehnice complete ale microserviciului de chat sunt prezentate în Capitolul 8.

4.5 Strategie de deployment și CI/CD

Sistemul KickSneak utilizează Docker pentru un deployment local unificat. Un fișier `docker-compose.yml` configurează cele 12 servicii, iar scripturile de bootstrap (`setup.ps1` pentru Windows și `setup.sh` pentru Linux/macOS) automatizează instalarea dependențelor și asamblarea containerelor.

Configurările de mediu sunt stocate într-un fișier `.env`. Cheile private Firebase Admin SDK sunt codificate Base64 și decodificate direct în memoria RAM a backend-ului pentru a preveni salvarea lor în istoricul Git.

Arhitectura este compatibilă cu integrările CI/CD (GitHub Actions/Azure DevOps). Fiecare serviciu folosește un Dockerfile bazat pe build-uri multi-stage (Alpine runtime),

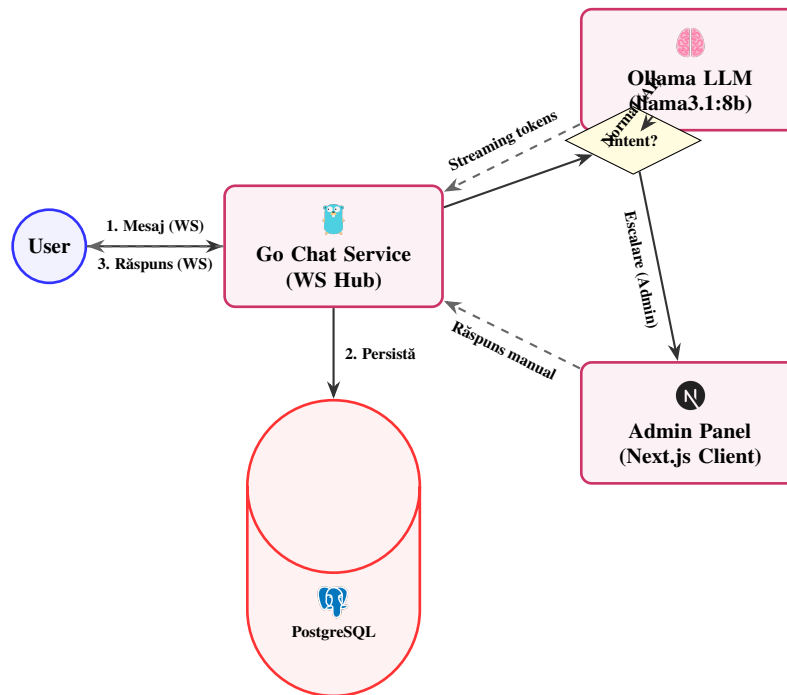


Figura 4.2: Schema fluxurilor conversaționale (User-LLM și escalare User-Admin) în micro-serviciul de chat

facilitând testarea automată și publicarea imaginilor în registry.

Componentă locală	Echivalent Azure Cloud	Justificare tehnică
Backend Core	Azure App Service / AKS	Scalare automată a API-urilor
Next.js Admin	Azure App Service / Vercel	Suport nativ pentru serverless rendering
PostgreSQL 16	Azure DB for PostgreSQL	Bază de date administrată cu backup automat
Redis 7	Azure Cache for Redis	Performanțe ridicate pentru licitații
Elasticsearch 8	Elastic Cloud on Azure	Serviciu administrat de căutare
Azurite (Blob/Table)	Azure Storage Account	Persistență reală de binare și date NoSQL
Kotlin Functions	Azure Functions (Kotlin)	Execuție serverless la evenimente asincrone

Tabela 4.2: Strategia de migrare a infrastructurii KickSneak în cloud-ul Azure

4.6 Persistență și Azure Storage

PostgreSQL 16 reprezintă sursa tranzacțională unică a platformei. Gestionează datele esențiale ale aplicației: utilizatorii și rolurile lor, catalogul principal de produse, comenzile, tranzacțiile financiare și sesiunile de chat.

Redis 7 stochează în memorie datele volatile de înaltă frecvență. Gestionează licitațiile active pentru a suporta bid-uri concurente, păstrează token-urile JWT blocate (blacklisted) și servește drept cache pentru a reduce numărul de interogări pe PostgreSQL.

Elasticsearch 8 stochează documentele JSON optimizate pentru interogări text. Indexează elementele de catalog (titlu, descriere, brand, taguri) pentru a oferi căutări full-text și sugestii live extrem de rapide.

Azure Storage (emulat prin Azurite) reține fișierele binare mari în Blob Storage (imagini de produse, fișiere din chat) și date nestructurate de volum mare în Table Storage (jurnalele de audit și istoricul securizat), degrevând baza relațională.

Capitolul 5

Securitate aplicată

5.1 Filozofia de securitate — Defense in Depth

Securitatea reprezintă un pilon fundamental în proiectarea platformei KickSneak. Sistemul adoptă principiul de securitate în profunzime (*Defense in Depth*), structurând protecția pe cinci straturi defensive independente capabile să se compenseze reciproc în cazul defectării uneia dintre ele:

1. **Stratul Edge / Periferic:** Gestionează securitatea la nivel de transport HTTP, forțarea HTTPS, reguli CORS și headere de securitate browser-side.
2. **Stratul de Rate Limiting:** Protejează API-ul împotriva atacurilor de tip DDoS și brute-force pe rutele sensibile prin limitări per IP sau utilizator.
3. **Stratul de Autentificare:** Validează criptografic identitatea utilizatorilor prin Firebase SDK, corelat cu blacklist în Redis și urmărirea IP-urilor.
4. **Stratul de Autorizare:** Asigură controlul permisiunilor prin maparea identității pe roluri în baza de date (RBAC) și impunerea politicii Row Level Security (RLS).
5. **Stratul de Protecție a Datelor:** Previne SQL Injection prin ORM-uri parametrizate (EF Core și Prisma) și liste albe (whitelists) în interogările Dapper brute.

Suplimentar, sistemul integrează un strat transversal de audit și observabilitate (TraceMiddleware), care urmărește fiecare tranzacție, alături de reguli de cod restrictive.

5.2 Pipeline-ul de middleware-uri

În cadrul backend-ului principal în .NET 10, toate request-urile HTTP sosite sunt trecute printr-un pipeline secvențial de middleware-uri configurate în `Program.cs`:

1. `SecurityHeadersMiddleware`: Setează headerele HTTP de securitate pe orice răspuns, inclusiv în caz de excepții.
2. `TraceMiddleware`: Generează un identificator unic (`TraceId`), pornește cronometrarea performanței și inițializează contextul de auditare.
3. `RateLimiterMiddleware`: Verifică frecvența request-urilor și respinge apelurile abuzive cu codul HTTP 429, prevenind consumul de CPU.
4. `FirebaseAuthMiddleware`: Extrage token-ul JWT din header, verifică validitatea semnăturii criptografice și populează `Claims` în `ClaimsPrincipal`.
5. `UserResolverMiddleware`: Folosește identitatea pentru a efectua un lookup rapid în DB și a popula contextul tranzacțional (`HttpContext`) cu ID-ul intern și rolul aferent (`DbRole`).
6. `Endpoint Handlers / Controllers`: Logica efectivă a aplicației care rulează într-un context complet autentificat și controlat.

Plasarea `RateLimiterMiddleware` înaintea `FirebaseAuthMiddleware` garantează că atacurile brute-force nu consumă resurse computaționale pentru validarea token-urilor JWT, request-urile abuzive fiind blocate instant pe baza adresei IP.

5.3 SecurityHeadersMiddleware

Rolul principal al `SecurityHeadersMiddleware` constă în întărirea politicilor de securitate din browserele clienților prin injectarea de headere HTTP specifice. Tabelul 5.1 prezintă headerele setate la nivelul fiecărui răspuns returnat de platformă. Headerele `Cache-Control` și `Pragma` sunt setate pe `no-store`, `no-cache` pentru a preveni stocarea datelor sensibile, în timp ce `Referrer-Policy` protejează datele de navigare.

Suplimentar, pentru a îngreuna analiza amprenteii software (*fingerprinting reduction*), middleware-ul elimină headerele implicite: `X-Powered-By`, `Server`, `X-AspNet-Version` și `X-AspNetMvc-Version`. Politica CSP (*Content Security Policy*) interzice explicit scripturile inline (`unsafe-inline`) și cele aduse de pe CDN-uri externe.

Header HTTP	Valoare configurată	Rol de securitate
X-Content-Type-Options	nosniff	Împiedică MIME sniffing, blocând execuția fișierelor script mascate ca imagini.
X-Frame-Options	DENY	Protecție anti-clickjacking; interzice încadrarea aplicației în iframe-uri externe.
Content-Security-Policy	default-src 'self'; ...	Permite execuția de scripturi și resurse doar de pe domeniul propriu, prevenind XSS.
Strict-Transport-Security	max-age=31536000	(HSTS) Forțează browserul să comunice exclusiv prin HTTPS securizat.
Permissions-Policy	geolocation=(), ...	Blochează accesul programatic la cameră, microfon și senzori hardware.

Tabela 5.1: Configurația de securitate a headerelor HTTP în KickSneak

5.4 RateLimiterMiddleware

Middleware-ul de limitare a ratei de acces (RateLimiterMiddleware) folosește un algoritm de fereastră glisantă (*sliding window*) implementat in-memory prin structuri thread-safe. Algoritmul reține timestamp-ul fiecărui request într-o coadă `Queue<DateTime>`. La sosirea unui request nou, procesul efectuează trei pași:

- 1. Curățarea:** Elimină din coada de apeluri timestamp-urile mai vechi decât lungimea ferestrei setate (ex. 60 de secunde).
- 2. Validarea:** Verifică dacă numărul de apeluri rămase depășește pragul maxim definit; în caz afirmativ, returnează HTTP 429.
- 3. Înregistrarea:** Adaugă timestamp-ul curent în coada utilizatorului și permite continuarea execuției request-ului.

Platforma aplică politici diferențiate în funcție de sensibilitatea rutei apelate, conform Tabelului 5.2.

Pentru utilizatorii autentificați, limita este multiplicată cu 3, iar cheia bucket-ului devine `{UserId}:{Path}`, în timp ce pentru vizitatorii anonimi se folosește `{IP}:{Path}`. Mecanismul utilizează un `ConcurrentDictionary` cu blocări exclusive (lock) pe cheie pentru a asigura thread-safety.

Categorie rute	Limită request-uri	Fereastră timp	Justificare securitate
/api/auth/*	10 request-uri	60 secunde	Protecție brute-force pe validări token / login.
/api/checkout/*	5 request-uri	60 secunde	Previne abuzul pe API-urile de plată și checkout Stripe.
/api/auctions/*/bid	30 request-uri	60 secunde	Toleranță la plasarea rapidă de bid-uri în ultimele secunde.
Rute implicite (Default)	100 request-uri	60 secunde	Limitare generală de trafic pentru utilizatori normali.

Tabela 5.2: Politicile de limitare a ratei de acces per rută în KickSneak

5.5 FirebaseAuthMiddleware

Autentificarea utilizatorilor în KickSneak este gestionată asincron prin integrarea Firebase Authentication SDK. Procesul din FirebaseAuthMiddleware se desfășoară după următorul algoritm:

1. Extrage header-ul Authorization din request; dacă acesta lipsește sau este invalid, conexiunea primește automat rolul de `ks_guest`.
2. Verifică blacklist-ul în Redis (`auth:blacklist:{token_prefix_20}`) pentru a invalida token-urile utilizatorilor deconectați (logout).
3. Validează criptografic token-ul JWT Firebase (semnătură, expirare, audiență) prin `VerifyIdTokenAsync`.
4. Extrage Claims de identitate (UID Firebase, e-mail, nume) și asociază request-ului un obiect `ClaimsPrincipal`.
5. **Audit și sesiune:** Monitorizează IP-ul în Redis (`auth:ips:{uid}`) emitând alerte la locații noi și actualizează sesiunea activă cu un TTL de o oră.

Dacă validarea criptografică eșuează, se interceptează excepția, se înregistrează eroarea pentru audit și se returnează codul HTTP 401 simplu, prevenind scurgerea datelor tehnice interne.

5.6 UserResolverMiddleware

Middleware-ul `UserResolverMiddleware` realizează maparea dintre identitatea externă Firebase și structurile interne din baza de date KickSneak, decuplând autentificarea de accesul la date:

1. Extrage identificatorul unic `uid` din obiectul `ClaimsPrincipal` populat anterior de middleware-ul Firebase.
2. Execută o interogare rapidă cu micro-ORM-ul `Dapper` direct pe baza de date PostgreSQL, folosind conexiunea administrativă:

```
1 SELECT u."Id",  
2         EXISTS(SELECT 1 FROM sellers s WHERE s."UserId" = u."  
3 Id" AND NOT s."IsDeleted") AS "IsSeller"  
4 FROM users u  
5 WHERE u."FirebaseUid" = @Uid AND NOT u."IsDeleted"
```

3. Populează contextul scoped `HttpContext` cu ID-ul intern (`UserId`) și rolul corespunzător (`DbRole.Seller` sau `DbRole.User`).
4. Salvează aceste valori în `HttpContext.Items` sub cheile `DbUserId` și `IsSeller` pentru acces rapid.

Pentru prima autentificare a unui utilizator nou pe platformă, `UserResolverMiddleware` setează rolul temporar de `DbRole.User` în loc de `DbRole.Guest`, permițând inserarea inițială în tabela `users` (altfel blocată de RLS). Lookup-ul prin `Dapper` evită overhead-ul EF Core, asigurând o latență sub o milisecundă, cu parametrizare strictă contra SQL Injection.

5.7 TraceMiddleware — Audit și observabilitate

La pornirea request-ului, `TraceMiddleware` generează sau extrage un identificator unic de request (`TraceId`), pe care îl injectează în răspuns ca `X-Trace-Id` și îl asociază logurilor. Middleware-ul folosește un `Stopwatch` pentru a cronometra performanța și, în caz de excepție necontrolată, înregistrează stack-trace-ul în Loki corelat cu `TraceId`. În final, calculează durata totală și înregistrează metricile în Prometheus prin interfața `IObservabilityFactory`.

5.8 Securitate la nivel de date

Securitatea la nivel de date garantează că baza de date nu poate fi compromisă prin interogări abuzive. Toate interogările EF Core și Prisma sunt traduse în interogări SQL parametrizate în mod nativ, eliminând SQL Injection. În porțiunile unde se utilizează Dapper pentru comanda SET LOCAL ROLE, valorile pentru rolul bazei de date sunt preluate dintr-o listă albă bazată pe structura unui enum:

```
1 string pgRole = role switch
2 {
3     DbRole.User => "ks_user",
4     DbRole.Seller => "ks_seller",
5     DbRole.Admin => "ks_admin",
6     DbRole.Chat => "ks_chat_service",
7     _ => "ks_guest"
8 };
```

Intrările din exterior nu influențează structura comenzii SQL. Politicile RLS acționează ca un ultim strat defensiv, refuzând returnarea datelor care aparțin altor utilizatori în caz de bug-uri.

5.9 Securitate la nivel de cod

Platforma aplică principiul privilegiului minim direct în organizarea fișierelor de cod prin modificatori de acces C# moderni:

- ▷ **Modificatorul sealed:** Clasele de middleware și serviciile sunt declared ca `public sealed class`, interzicând moștenirea și suprascrierea.
- ▷ **Modificatorul internal:** Utilizat pe clasele auxiliare care prelucrează date sensibile din proiectul de infrastructură.
- ▷ **file-scoped types:** Declară tipuri (cum ar fi clasa de lookup `UserLookup`) ca `file class`, fiind invizibile în afara fișierului de cod.
- ▷ **Imutabilitatea:** Setările de securitate utilizează `record` și proprietăți de tip `init`, prevenind modificarea politicilor la runtime.
- ▷ **Constructorii primari (Primary Constructors):** Declară automat dependențele de securitate injectate (logger, servicii Redis) ca `readonly`.

Capitolul 6

Backend .NET 10

6.1 Clean Architecture aplicată

Implementarea nucleului tranzacțional respectă organizarea Clean Architecture. Proiectul este structurat într-o soluție .NET (KickSneak.sln) cu cinci subproiecte distincte: Domain (entități pure), Application (cazuri de utilizare și abstracții), Persistence (EF Core și repozitorii), Infrastructure (Stripe, Redis, Firebase) și WebApi (Minimal APIs și DI). Inversarea dependențelor este realizată prin înregistrarea implementărilor concrete în containerul DI la runtime.

Pentru configurări Strongly-Typed, setările din `appsettings.json` sunt mapate pe clase concrete C# prin pattern-ul *ConfigurableObjects*:

```
1 builder.Services.Configure<StripeSettings>(
2     builder.Configuration.GetSection(nameof(StripeSettings)));
3 builder.Services.AddSingleton(sp =>
4     sp.GetRequiredService<IOptions<StripeSettings>>().Value);
```

Sistemul optimizează timpii de pornire prin injectarea leneșă (*Lazy<T>*), instanțiind dependențele complexe doar la prima interogare. Toate dependențele sunt injectate ca *readonly* prin constructori primari (*Primary Constructors*), asigurând imutabilitatea referințelor.

6.2 Modelul de date — Entități și relații

Modelul de date relațional din KickSneak este normalizat și structurat pe șase mari domenii funcționale:

- ▷ **Catalog:** Entitatea centrală Product (preț, retail, flag-ul IsAuction) este legată de tabelele de atribute (Brand, Category, Color) și de stocurile fizice (StockItem, Size).
- ▷ **Users & Sellers:** User (identificat prin FirebaseUid) se află în relații cu Address și o relație opțională 0 : 1 cu Seller (rating, stare verificare).
- ▷ **Commerce:** Conține entitățile pentru fluxul de plată și checkout: Cart, CartItem, Order, OrderItem, Payment (Stripe) și Offer.
- ▷ **Auctions:** Agregatul Auction controlează tranzacțiile de licitație, având o relație 1 : N cu ofertele depuse (Bid) și regulile setate (AutoBid).
- ▷ **Notifications & Chat:** Gestionează logica de mesagerie prin entitățile Notification, ChatSession și ChatMessage (roluri: user/assistant/admin).
- ▷ **Favorites & Returns:** Tabelele de legătură Favorite și Return pentru gestionarea listei de dorințe și a retururilor.

Configurarea schemei PostgreSQL se face în stratul de Persistence prin Fluent API, utilizând denumiri PascalCase case-sensitive și indecși adecvați pe cheile străine.

6.3 RBAC și RLS — implementare Unit of Work

Punctul central de coordonare a politicilor RLS este clasa UnitOfWork. Pentru a asigura atomicitatea operațiunilor de scriere multiple sub regulile de securitate active, se utilizează metoda `ExecuteInTransactionAsync`, care gestionează deschiderea tranzacției, aplicarea RLS, salvarea modificărilor și rollback-ul automat în caz de excepție:

```
1 public async Task ExecuteInTransactionAsync(Func<Task> action,
2     CancellationToken ct = default)
3 {
4     await using var transaction = await context.Database.
5         BeginTransactionAsync(ct);
6
7     try
8     {
9         await ApplyRlsAsync(ct);
10
11         await action();
12
13         await context.SaveChangesAsync(ct);
14
15         await transaction.CommitAsync(ct);
16     }
```

```
14     }
15     catch
16     {
17         await transaction.RollbackAsync(ct);
18         throw;
19     }
20 }
```

Metoda privată `ApplyRlsAsync(ct)` interoghează starea din `RlsContext` și determină rolul ce trebuie aplicat conexiunii printr-o structură switch bazată pe o listă albă:

```
1 private async Task ApplyRlsAsync(CancellationTokentoken ct)
2 {
3     if (rlsContext.Role == DbRole.Guest)
4         return;
5
6     var role = rlsContext.Role switch
7     {
8         DbRole.User => "ks_user",
9         DbRole.Seller => "ks_seller",
10        DbRole.Admin => "ks_admin",
11        _ => (string?)null
12    };
13
14    if (role is null) return;
15
16    var conn = context.Database.GetDbConnection();
17
18    if (conn.State != System.Data.ConnectionState.Open)
19        await conn.OpenAsync(ct);
20
21    await conn.ExecuteAsync(
22        $"SET LOCAL ROLE {role}; SELECT set_config('app.
23        current_user_id', @UserId, true);",
24        new { UserId = rlsContext.UserId }
25    );
26 }
```

Utilizarea instrucțiunii `SET LOCAL` garantează că rolul stabilit este valabil strict pe durata tranzacției, eliminând riscul scurgerii de privilegii în pool-ul de conexiuni bazei de date.

6.4 Sistemul de licitații — Redis-first

Licitarea în timp real este implementată printr-un pattern de tip *Redis-first* pentru a evita blocajele de tranzacții (*deadlocks*) în PostgreSQL sub trafic intens. Starea licitațiilor este stocată în Redis prin Hash-uri (`auction:{id}` pentru starea curentă și `auction:{id}:autobids` pentru bidding automat), Liste (`auction:{id}:bids` pentru istoric) și un Sorted Set (`auctions:active` ordonat după data finalizării).

Pentru a evita scrierile concurente eronate, plasarea unui bid folosește optimistic locking prin comenzile WATCH, MULTI și EXEC. Algoritmul detaliat de plasare a unui bid și auto-bidding recursiv este prezentat în Algoritmul 1.

Algorithm 1: Algoritmul de plasare bid cu control optimist al concurenței în Redis

Input: auctionId, userId, bidAmount
Output: ServiceResult (Succes sau Eroare)

```

1 key ← "auction:" + auctionId
2 WATCH key
3 currentBid ← HGET key "currentBid"
4 endsAt ← HGET key "endsAt"
5 if currentTime ≥ endsAt then
6   | UNWATCH
7   | return Error("Licitația a expirat")
8 end
9 if bidAmount ≤ currentBid then
10  | UNWATCH
11  | return Error("Suma introdusă este prea mică")
12 end
13 MULTI
14 HSET key "currentBid" bidAmount
15 HSET key "currentBidderId" userId
16 LPUSH key + ":bids" bidAmount
17 result ← EXEC
18 if result is null then
19  | return PlaceBidAsync(auctionId, userId, bidAmount) // Retry recursiv
20 end
21 Publish BidPlacedEvent(auctionId, bidAmount, userId)
   la SignalR AuctionsHub
22 TriggerAutoBidChain(auctionId, bidAmount, userId)
23 return Success()
```

Sincronizarea datelor se face prin jobul Quartz.NET AuctionSyncJob (care scrie datele din Redis în PostgreSQL la fiecare 5 minute) și AuctionCloseJob (care finalizează licitația la expirarea timpului și generează comanda).

6.5 Integrare Stripe

Plățile sunt gestionate prin Stripe Checkout API pentru conformitate PCI-DSS. Serviciul CheckoutService inițializează sesiunea Stripe trimițând produsele, costurile și metadatele de identificare:

```
1 var options = new SessionCreateOptions
2 {
3     PaymentMethodTypes = new List<string> { "card" },
4     LineItems = items,
5     Mode = "payment",
6     SuccessUrl = "https://kicksneak.ro/checkout/success?session_id={CHECKOUT_SESSION_ID}",
7     CancelUrl = "https://kicksneak.ro/checkout/cancel",
8     Metadata = new Dictionary<string, string>
9     {
10         { "OrderId", orderId.ToString() },
11         { "UserId", userId.ToString() }
12     }
13 };
14 var service = new SessionService();
15 Session session = await service.CreateAsync(options);
```

Confirmarea se face asincron prin endpoint-ul de webhook /webhooks/stripe, unde se validează semnătura digitală a request-ului folosind WebhookSecret. Evenimentul checkout.session.completed modifică statusul comenzii în PostgreSQL ca fiind plătită (Paid) și trimite notificarea prin SignalR.

6.6 Minimal APIs și organizarea endpoint-urilor

Rutele API sunt organizate utilizând interfața IEndpointGroup pentru a menține curățenia în Program.cs. De exemplu, endpoint-urile de licitații sunt definite în AuctionEndpoints.cs:


```
1 public class AuctionEndpoints : IEndpointGroup
2 {
3     public void Map(WebApplication app)
4     {
5         var group = app.MapGroup("/api/v1/auctions");
6
7         group.MapGet("/", GetActiveAuctions).AllowAnonymous();
8         group.MapGet("/{id}", GetAuctionById).AllowAnonymous();
9         group.MapPost("/{id}/bid", PlaceBid).RequireAuthorization();
10        group.MapPost("/{id}/autobid", SetAutoBid).
11        RequireAuthorization();
12    }
13 }
```

La pornire, o metodă de extensie înregistrează automat toate implementările `IEndpointGroup` sub prefixul `/api/v1/`. Logica întoarcere rezultate încapsulate în `ServiceResult<T>`, facilitând maparea pe status coduri HTTP standard.

6.7 Background Services — IHostedService și Channels

Managementul sarcinilor asincrone de fundal este realizat prin servicii de fundal (`BackgroundService`) în corelație cu primitivele `Channel<T>`. Spre deosebire de o coadă simplă care necesită polling periodic, `Channel<T>` oferă un model de așteptare asincron neblokant (`ReadAllAsync()`), reducând consumul CPU la zero când canalul este gol.

În platformă sunt active două servicii de fundal bazate pe acest pattern:

1. **ElasticsearchSyncService:** Ascultă pe `Channel<ElasticSyncEvent>` pentru a propaga modificările produselor în indexul Elasticsearch în mod asincron.
2. **NotificationDispatchService:** Ascultă pe `Channel<NotificationEvent>` pentru a notifica utilizatorii în timp real prin SignalR.

Algoritmul de execuție din interiorul worker-ului asincron de sincronizare Elasticsearch este detaliat în Algoritmul 2.

Algorithm 2: Metoda `ExecuteAsync` din worker-ul de sincronizare asincronă Elasticsearch

Input: `CancellationToken ct`

```
1 while not ct.IsCancellationRequested do
2   await foreach (var evt in _channel.Reader.ReadAllAsync(ct)) productId ←
     evt.ProductId
3   opType ← evt.OperationType
4   if opType == OperationType.Upsert then
5     product ← await _db.GetProductForIndexAsync(productId)
6     await _elasticClient.IndexAsync(product, ct)
7   end
8   if opType == OperationType.Delete then
9     await _elasticClient.DeleteAsync(productId, ct)
10  end
11  OperationCanceledException Break // Shutdown graceful
12  Exception ex LogError("Eroare la indexare Elasticsearch", ex)
13  await Task.Delay(5000, ct) // Așteptare înainte de reîncercare
14 end
```

Capitolul 7

Interfața client (React)

Aplicația frontend destinată utilizatorilor finali ai platformei KickSneak joacă un rol critic în succesul sistemului, fiind responsabilă direct de oferirea unei experiențe de navigare rapide și interactive. Aceasta este dezvoltată utilizând biblioteca React 18, TypeScript și sistemul de build Vite. În acest capitol sunt prezentate structura componentelor client, gestiunea stării globale prin Zustand, optimizarea regăsirii datelor și integrarea fluxurilor în timp real.

7.1 Arhitectură și principii de design

Interfața grafică este structurată în conformitate cu metodologia *Atomic Design* [7], fiind descompusă în elemente ierarhice: **Atoms** (componente de bază fără stare de business, ex. `GlassButton`), **Molecules** (asamblări simple cu stări interne de UI, ex. `SearchBar`), **Organisms** (secțiuni complexe ce interacționează cu store-urile globale, ex. `Navbar`) și **Pages** (ecrane complete rulate sub formă de rute, ex. `AuctionDetailPage`). Stilizarea componentelor utilizează *CSS Modules* (`ComponentName.module.css`) asociate fiecărei componente, prevenind coliziunile. Designul vizual adoptă o estetică de tip *Glassmorphism* (transparență opacă, border-uri fine și filtre de blur), configurată prin variabile CSS globale. Pentru rularea fără backend, s-a implementat variabila `VITE_USE MOCKS=true`, care redirectionează apelurile către mock-uri JSON statice.

7.2 Gestiunea stării cu Zustand

Gestiunea stării globale utilizează biblioteca Zustand [18], bazată pe hook-uri React strongly-typed, care elimină boilerplate-ul specific Redux și previne re-randările inutile prin subscripții granulare. Store-urile configurate includ:

- ▷ `useAuthStore`: Reține datele de profil Firebase, token-ul JWT și starea sesiunii.
- ▷ `useCartStore`: Administrează produsele din coș și calculează prețul în timp real.
- ▷ `useFavoritesStore`: Gestionează produsele marcate ca favorite.
- ▷ `useNotificationStore`: Stochează alertele primite în timp real prin SignalR.

Store-urile de coș și favorite folosesc **Optimistic Updates**: starea se modifică instantaneu în interfață, urmată de apelul API asincron, executându-se un rollback automat în caz de eșec. Starea filtrelor active pe paginile de rezultate este păstrată direct în query parameters din URL (ex. `?brand=Nike&size=42`), permițând navigarea nativă a browserului.

7.3 Serviciul `CachedFetchService`

Pentru a reduce latențele de încărcare a datelor statice, s-a dezvoltat `CachedFetchService`, care aplică o strategie *stale-while-revalidate* peste `localStorage`:

1. Verifică datele salvate local și le returnează instantaneu către componentă printr-un callback (`onData`) pentru randarea fără loading spinners.
2. Lansează o cerere HTTP asincronă în fundal către server pentru a prelua datele actuale.
3. Compară rezultatele și, dacă există diferențe, actualizează memoria locală și apelează din nou callback-ul `onData` pentru re-randarea lină.

7.4 Pagini și fluxuri interactive

Pagina de căutare (`SearchResultsPage`) utilizează tehnica de debouncing la introducerea textului pentru a evita suprasolicitarea API-ului și implementează *infinite scroll* prin API-ul nativ `IntersectionObserver`.

Pagina detaliată a licitației (`AuctionDetailPage`) integrează un countdown timer

reactiv și un mecanism de *anti-snipe*: plasarea unui bid în ultimele 2 minute extinde automat timpul de finalizare cu 1 minut, starea fiind sincronizată prin SignalR.

Tabloul de bord al profilului este un tablou de bord complet al profilului cu secțiuni pentru comenzi, adrese, retururi, chat și funcționalități dedicate vânzătorilor.

7.5 Comunicarea în timp real și optimizări

Sincronizarea licitațiilor folosește biblioteca client @microsoft/signalr pentru a asculta evenimentul BidPlaced și a actualiza starea în mod reactiv:

```
1 connection.on("BidPlaced", (data: BidPlacedPayload) => {  
2   if (data.auctionId === currentAuctionId) {  
3     updateAuctionState(data.amount, data.bidderId);  
4   }  
5 });
```

Pentru suport, clientul utilizează o conexiune WebSocket nativă către Go Chat Service. Token-urile generate de modelul Ollama sunt citite în format streaming și afișate progresiv cu un efect vizual de tastare.

Performanța este optimizată prin importul asincron al rutelor utilizând `React.lazy()` și `Suspense` pentru fragmentarea bundle-ului principal, utilizarea hook-ului `useMemo` pentru a preveni recalcularea filtrelor pe liste mari, și aplicarea listener-ului `useWindowSize` pentru adaptarea ecranului fără randări redundante în DOM.

Capitolul 8

Microserviciul de chat (Go)

Pentru a asigura o asistență conversațională eficientă și rapidă în timp real, am proiectat și implementat un microserviciu dedicat pentru chat, scris în limbajul Go. Acest serviciu se conectează cu modelul LLM local rulate prin Ollama și facilitează comunicarea bidirecțională directă prin protocoale WebSocket. Alegerea limbajului Go și separarea acestei funcționalități într-un serviciu independent au la bază considerente clare de performanță, concurență și decuplare a responsabilităților arhitecturale. În acest capitol este detaliat design-ul tehnic al modulului, integrarea cu modelul mare de limbaj Llama 3.1 și modul în care se gestionează tranzacțiile de date și procesele de escaladare la operatorii umani.

8.1 Motivația alegerii Go pentru microserviciul de chat

Aplicațiile de chat în timp real se caracterizează printr-un comportament de tip *I/O-bound*, marcat de menținerea unui număr foarte mare de conexiuni deschise în mod persistent (WebSocket). În limbajele tradiționale sau framework-urile sincrone, fiecare conexiune persistentă necesită alocarea unui fir de execuție al sistemului de operare (*OS Thread*), consumând între 1 și 8 MB per thread, ceea ce limitează scalabilitatea serverului.

Limbajul Go rezolvă această limitare prin intermediul modelului său de concurență bazat pe *goroutine*-uri [2]. O *goroutine* este administrată la nivel de runtime, stiva sa inițială ocupând doar 2 KB de memorie și extinzându-se dinamic. De asemenea, modelul CSP (*Communicating Sequential Processes*) utilizează canale (*channels*) pentru transmiterea sigură a mesajelor între *goroutine*-uri fără lock-uri complexe, fiind ideal pentru streaming-ul asincron al token-urilor de la Ollama. Din punct de vedere operațional, compilatorul generează un

singur binar static de dimensiuni reduse (15 MB în Docker), asigurând o modularitate completă și izolarea resurselor față de restul platformei.

8.2 Arhitectura serviciului

Structura proiectului reflectă organizarea modulară standard din ecosistemul Go, fiind separată în pachete cu responsabilități clare:

- ▷ `cmd/server/main.go`: Punctul de intrare, responsabil cu încărcarea configurațiilor, inițializarea PostgreSQL și pornirea serverului.
- ▷ `internal/config`: Încarcă variabilele de mediu (string PostgreSQL, URL Ollama) într-o structură strongly-typed.
- ▷ `internal/domain`: Conține modelele de date comune (`ChatSession`, `ChatMessage`) și payload-urile JSON pentru WebSocket.
- ▷ `internal/repository`: Implementează citirea și scrierea directă în PostgreSQL folosind driverul `pgx/v5`.
- ▷ `internal/service`: Coordonarea logică realizată de `OllamaService` și `ChatService`.
- ▷ `internal/handler`: Gestionează handler-urile HTTP și upgrade-ul conexiunilor la WebSocket via `gorilla/websocket`.

Configurația aplicației se realizează exclusiv prin variabile de mediu. Serverul expune endpoint-urile `/ws/user-chat` pentru clienți și `/ws/seller-chat` pentru vânzători.

8.3 Integrarea Ollama cu streaming

Punctul central al inteligenței conversaționale a asistentului KickSneak îl reprezintă conexiunea cu modelul *llama3.1:8b* (sau modelul *qwen2.5:7b*), rulat local prin Ollama. Pentru a evita latența, am implementat streaming-ul token-by-token.

Când un utilizator trimite un mesaj, `ChatService` parcurge următorul flux:

1. Salvează mesajul utilizatorului în PostgreSQL și apelează `AnalyzeIntent` pentru a clasifica intenția utilizatorului prin prompt-ul de sistem.

2. În cazul în care interogarea necesită date din DB, rulează interogarea rapidă în PostgreSQL și injectează rezultatele ca mesaj sistem, apoi lansează cererea HTTP POST către Ollama cu "stream": true.

Răspunsul stream este citit linie cu linie utilizând structura bufio.Scanner pe corpul de răspuns HTTP, conform algoritmului descris în Algoritmul 3.

Algorithm 3: Algoritmul de streaming token-by-token de la Ollama API în Go

Input: ctx, messages, prompt, onTokenCallback
Output: Răspunsul complet asamblat ca string

```

1 reqBody ←
  Marshal(OllamaRequest{Model: model, Messages: messages, Stream: true})
2 req ← NewRequest("POST", baseURL + "/api/chat", reqBody)
3 resp ← client.Do(req)
4 resp.Body.Close()
5 if resp.StatusCode ≠ 200 then
6   | return Error("Status " + resp.StatusCode)
7 end
8 fullResponse ← ""
9 scanner ← bufio.NewScanner(resp.Body)
10 while scanner.Scan() do
11   line ← scanner.Bytes()
12   varchunk OllamaStreamChunk
13   if Unmarshal(line, &chunk) == nil then
14     token ← chunk.Message.Content
15     if token ≠ "" then
16       | fullResponse ← fullResponse + token
17       | onTokenCallback(token)
18     end
19     if chunk.Done then
20       | Break
21     end
22   end
23 end
24 return fullResponse

```

Pentru fiecare bucată de text recepționată, se apelează funcția onTokenCallback, care o expediază clientului prin WebSocket sub forma unui payload "token". La finalizarea stream-ului, mesajul integral este salvat în PostgreSQL.

8.4 Fluxul de escaladare la operator uman

Asistentul își recunoaște limitele de înțelegere. În cadrul metodei `AnalyzeIntent`, modelul LLM clasifică mesajul utilizatorului utilizând un format de tip JSON:

```
1 {  
2   "intent": "escalate",  
3   "tables": [],  
4   "limit": 0  
5 }
```

Modelul returnează intenția "escalate" atunci când identifică reclamații, fraudă, litigii financiare sau solicitări explicite de conectare cu un om.

În momentul în care clasificatorul întoarce valoarea "escalate", `ChatService` execută următoarele operațiuni automate:

1. Trimite utilizatorului un mesaj politicos de redirectare: „Am înțeles că doriți să discutați cu un operator. Vă conectez imediat la echipa noastră de suport.”
2. Actualizează starea sesiunii curente ca fiind escaladată în PostgreSQL:

```
1   UPDATE chat_sessions SET escalated = true, status = '  
   escalated' WHERE id = $1;  
2
```

3. Transmite un payload specific prin WebSocket, determinând clientul React să blocheze input-ul către LLM și să afișeze indicatorul de conectare cu operatorul.

Sesiunile escalate devin vizibile în panoul administrativ Next.js, fiind preluate din baza de date prin interogarea rutei `GET /api/chat/admin/sessions?escalated=true`. Răspunsurile scrise de operator în interfața administrativă sunt trimise prin intermediul unui API HTTP securizat la serviciul Go, care le redirecționează către utilizator prin canalul WebSocket persistent deschis anterior.

8.5 Persistența sesiunilor și mesajelor în PostgreSQL

Securitatea la nivel de date este menținută prin configurarea unui rol dedicat în PostgreSQL numit `ks_chat_service`. Acest utilizator are permisiuni strict limitate la nivel de tabel: poate efectua operațiuni de tip `SELECT readonly` pe tabelele de catalog și are drepturi

complete de tip INSERT, SELECT și UPDATE exclusiv pe tabelele de chat (chat_sessions și chat_messages). Această abordare respectă principiul *least privilege*.

Interacțiunea cu baza de date folosește biblioteca pgx/v5 în mod direct, fără un ORM intermediar, asigurând performanțe optime și legarea parametrilor (de tipul \$1, \$2) pentru a preveni SQL injection. Scanarea rezultatelor se face în mod pozițional explicit:

```
1 err := row.Scan(&session.ID, &session.UserID, &session.ChatType, &  
    session.Status, &session.Escalated)
```

Sistemul implementează un mecanism de reconectare transparentă. La inițierea conexiunii, handlerul apelează GetOrCreateSession. Dacă există o sesiune activă, se reîncarcă istoricul complet al mesajelor și se trimite clientului pentru reluarea discuției. Titlul sesiunii este actualizat automat la primul mesaj trimis, fiind trunchiat la 60 de caractere.

8.6 Deployment și comunicare cu backend-ul .NET

Microserviciul de chat este complet containerizat utilizând un Dockerfile optimizat multi-stage:

```
1 # Stage 1: Build binary  
2 FROM golang:1.23-alpine AS builder  
3 WORKDIR /app  
4 COPY go.mod go.sum ./  
5 RUN go mod download  
6 COPY . .  
7 RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-w -s" -o chat-server  
    cmd/server/main.go  
8  
9 # Stage 2: Minimal runtime  
10 FROM alpine:3.19  
11 WORKDIR /app  
12 COPY --from=builder /app/chat-server .  
13 EXPOSE 8080  
14 CMD ["/chat-server"]
```

Utilizarea flag-urilor de compilare -w -s elimină informațiile de debugging, reducând dimensiunea binarului cu 40%, iar imaginea de runtime bazată pe Alpine asigură o suprafață de atac minimă și un consum redus de resurse.

Comunicarea dintre microserviciul Go și backend-ul principal .NET este proiectată ca o comunicare *indirectă*, realizată exclusiv prin intermediul bazei de date PostgreSQL partajate. Backend-ul .NET nu apelează niciodată direct endpoint-urile Go, fapt care decuplează complet disponibilitatea celor două servicii. Aplicația client din React deschide direct canalul WebSocket către portul serviciului Go (8080). Această topologie reduce latența la streaming prin eliminarea unui proxy intermediar prin serverul .NET, permițând livrarea directă a token-urilor generate de modelul Ollama către browser-ul utilizatorului.

Capitolul 9

Sistem de recomandare AI (Python + PyTorch)

Diferențierea pe piață a platformelor e-commerce moderne se realizează în mare parte prin relevanța recomandărilor oferite utilizatorilor. Pentru a depăși limitările sistemelor bazate pe popularitatea generală a produselor, am implementat în platforma KickSneak un modul de recomandări personalizate bazat pe inteligență artificială. Acesta rulează ca un microserviciu scris în Python, utilizând biblioteca PyTorch pentru definirea și antrenarea unei rețele neuronale de tipul *Neural Collaborative Filtering* (NCF). În acest capitol sunt prezentate fundamentele teoretice ale algoritmului, arhitectura rețelei, pipeline-ul de date și integrarea cu nucleul tranzacțional al aplicației.

9.1 Motivația și alegerea algoritmului

Sistemele de e-commerce se confruntă frecvent cu absența datelor de feedback explicit, bazându-se pe feedback implicit (vizualizări, favorite, achiziții) [10]. Algoritmii clasici precum SVD sau KNN folosesc produsul scalar, o funcție liniară care limitează capacitatea de a învăța interacțiuni complexe non-liniare. De aceea, am implementat modelul *Neural Collaborative Filtering* (NCF) [9], care înlocuiește produsul scalar cu o rețea neuronală profundă ce mapează utilizatorii și produsele într-un spațiu latent comun. Am ales biblioteca PyTorch pentru implementarea rețelei datorită modelului său de graf de calcul dinamic, a ușurinței de depanare și a compatibilității native cu instrumentele de prelucrare a datelor din ecosistemul Python (pandas, numpy, scikit-learn). Modulul AI este integrat în KickSneak sub forma unei secțiuni „Recomandate pentru tine” pe pagina principală și ca un filtru de reordonare a rezultatelor căutării, aducând pe primele poziții articolele cu scorul de afinitate cel mai mare pentru utilizatorul curent.

9.2 Arhitectura rețelei neuronale

Rețeaua neuronală NCF implementată primește ca intrare doi identificatori întregi: `user_id` și `product_id`. Aceștia sunt trecuți prin două straturi de embedding separate, care funcționează ca tabele de căutare a vectorilor latenți densi:

- ▷ **User Embedding:** O matrice de dimensiune $N_{users} \times d$, unde $d = 64$ reprezintă dimensiunea spațiului latent ales.
- ▷ **Product Embedding:** O matrice de dimensiune $N_{products} \times d$.

Dimensiunea latentă de 64 a fost aleasă pentru a asigura un echilibru optim între capacitatea de expresivitate a modelului și prevenirea supra-adaptării (*overfitting*) pe setul de date disponibil.

Vectorii rezultați din cele două straturi de embedding, e_u și e_i , sunt concatenați într-un singur vector de 128 de dimensiuni. MLP-ul este structurat pe trei straturi: 128→128 (BatchNorm1d + ReLU + Dropout 20%), 128→64 (BatchNorm1d + ReLU + Dropout 10%) și 64→1 (scor brut). Scorul brut este trecut printr-o funcție de activare Sigmoid, care maptează rezultatul în intervalul $[0, 1]$, reprezentând probabilitatea matematică ca utilizatorul să aibă o interacțiune pozitivă cu produsul evaluat.

Funcția de pierdere utilizată este *Binary Cross-Entropy* (BCE), definită ca:

$$L = -\frac{1}{N} \sum_{k=1}^N [y_k \log(p_k) + (1 - y_k) \log(1 - p_k)] \quad (9.1)$$

unde $y_k \in \{0, 1\}$ este eticheta reală de interacțiune, iar $p_k \in [0, 1]$ este probabilitatea prezisă de rețea. Optimizarea ponderilor se face prin algoritmul *Adam* cu o rată de învățare $\eta = 0.001$ și o penalizare L2 (*weight_decay*) de 10^{-5} . Pentru a antrena modelul pe feedback implicit, se utilizează tehnica de *negative sampling*: pentru fiecare interacțiune reală se generează în mod aleator 4 perechi negative (utilizator, produs cu care acesta nu a interacționat deloc), etichetate cu 0. Această rație de 1:4 asigură stabilitatea antrenării.

9.3 Pipeline-ul de date

Pipeline-ul de antrenare rulează asincron ca un script de fundal. Datele brute de interacțiune sunt extrase direct din PostgreSQL utilizând driverul *psycopg2* prin intermediul unui query de tip JOIN:

```
1 SELECT user_id, product_id, interaction_type FROM (  
2     SELECT "UserId" AS user_id, "ProductId" AS product_id, 'order' AS  
        interaction_type FROM orders  
3 UNION ALL  
4     SELECT "UserId", "ProductId", 'bid' FROM bids  
5 UNION ALL  
6     SELECT "UserId", "ProductId", 'favorite' FROM favorites  
7 ) ALL_INTERACTIONS;
```

Rezultatele sunt încărcate în pandas, unde interacțiunile primesc ponderi diferite (vizualizare: 1, favorite: 2, bid: 3, comandă: 5) și sunt binarizate. Identificatorii GUID sunt convertiți în indici prin LabelEncoder din scikit-learn, salvat ulterior în fișiere pickle pentru inferență. Setul de date este împărțit 80/20 train/validation și încapsulat în PyTorch Dataset și DataLoader (batch size 512, shuffle).

9.4 Training și evaluare

Procesul de antrenare rulează pe o durată de 20 de epoci. La finalul fiecărei epoci, modelul este evaluat pe setul de validare. Am implementat o logică de *early stopping* care monitorizează pierderea de validare: dacă aceasta nu înregistrează o scădere timp de 3 epoci consecutive, antrenarea este oprită prematur, fiind salvată cea mai bună stare a parametrilor în fișierul `ncf_model.pth`.

Evaluarea performanței sistemului folosește două metrice standard în sistemele de recomandare [23]:

- ▷ **Hit Rate@10** (HR@10): Verifică dacă produsul pe care utilizatorul l-a achiziționat se regăsește printre primele 10 recomandări generate de model.
- ▷ **Normalized Discounted Cumulative Gain@10** (NDCG@10): Evaluează calitatea poziționării recomandărilor relevante.

Pentru setul de date generat prin seed, am obținut valori ilustrative de $\text{HR@10} \approx 0.72$ și $\text{NDCG@10} \approx 0.48$. Deși rezultatele confirmă corectitudinea implementării arhitecturale, trebuie asumat că performanța pe un dataset real din producție ar fi diferită.

9.5 Servirea modelului via Flask

Modelul antrenat este servit printr-un API HTTP minimalist implementat cu framework-ul Flask. Fișierul de parametri `ncf_model.pth` și obiectele de encodare sunt încărcate în memoria RAM o singură dată, la pornirea containerului Python, în cadrul unei instanțe singleton.

Ruta principală de predicție este expusă la adresa `POST /api/recommend`. Corpul cererii conține identificatorul Firebase al utilizatorului și numărul maxim de recomandări solicitate (`top_k`):

```
1 {  
2   "user_id": "firebase_uid_exemplu",  
3   "top_k": 10  
4 }
```

La primirea cererii, serviciul Flask urmează pașii:

1. Convertește identificatorul Firebase în index; în caz de cont nou (cold start), returnează direct cele mai populare produse.
2. Generează un lot de perechi între indexul utilizatorului și toți indicii produselor active din catalog.
3. Rulează inferența (`model.eval()` sub contextul `torch.no_grad()`), ordonează rezultatele descrescător după scor și returnează identificatorii JSON.

Endpoint-ul de monitorizare `GET /api/health` returnează statusul intern al modelului, fiind utilizat de Docker pentru a valida starea de funcționare a serviciului.

9.6 Integrare cu backend-ul .NET și fallback

Integrarea cu nucleul platformei se realizează la nivel de server. Serviciul `RecommendationService` din backend-ul .NET apelează API-ul Flask prin intermediul unui client HTTP înregistrat în DI container cu o politică de timeout de 2 secunde.

În cazul în care microserviciul Python este offline sau depășește limita de timp alocată din cauza supraîncărcării, backend-ul .NET aplică o strategie de degradare grațioasă (*graceful degradation*), executând un query SQL de fallback direct în PostgreSQL:

```
1 SELECT "Id" FROM products ORDER BY "OrdersCount" DESC LIMIT @Limit;
```

Astfel, utilizatorul primește o listă stabilă cu cele mai populare produse ale momentului, asigurând continuitatea operării platformei.

Capitolul 10

Search, observabilitate și servicii de comunicare

Sistemele software de nivel enterprise necesită componente solide pentru căutarea rapidă a informațiilor, monitorizarea stării resurselor și comunicarea asincronă cu utilizatorii. Platforma KickSneak adresează aceste cerințe prin integrarea motorului de căutare dedicat Elasticsearch, a unui stack complet de observabilitate format din Prometheus, Loki și Grafana, precum și a unui serviciu poliglot de notificări scris în limbajul Kotlin sub formă de serverless functions. În acest capitol este descris modul în care aceste piese colaborează pentru a asigura performanța, diagnosticarea rapidă și informarea utilizatorilor în timp real.

10.1 Elasticsearch — indexare și căutare

Căutarea textuală rapidă este asigurată de Elasticsearch 8.x [4], un motor de căutare distribuit construit pe Apache Lucene. În KickSneak, am creat indexul `products` pentru stocarea sneakers-ilor disponibili. Schema indexului definește câmpurile `text` (`name`, `description`) pentru full-text search, `keyword` (`brand`, `category`, `tags`, `sizes`) pentru filtrare exactă și `float` (`price`) pentru range queries.

Pentru a asigura consistența datelor fără a penaliza performanța tranzacțională din backend, am implementat un model de sincronizare eventuală asincronă. Când un administrator sau un seller adaugă ori modifică un produs, serviciul `ProductService` introduce un eveniment de tip `ElasticSyncEvent(ProductId, OperationType)` într-un canal in-process asincron (`Channel<T>`) [14]. Un serviciu de fundal persistent (`ElasticsearchSyncService`, de tip `IHostedService`) ascultă acest canal în mod neblokant. Când primește un eveniment, serviciul preia starea actualizată a produsului din PostgreSQL, o transformă într-un document

JSON optimizat și o indexează în Elasticsearch.

Cererea de căutare din interfață apelează endpoint-ul de backend, care traduce parametrii într-o interogare Elasticsearch Query DSL. Căutarea folosește o structură de tip `multi_match` pe câmpurile `name`, `brand` și `tags`, aplicând un algoritm de căutare aproximativă prin setarea `"fuzziness": "AUTO"` [4]. Aceasta oferă o toleranță la greșeli de tipar (distanța Levenshtein de maximum 2 caractere), ordonând rezultatele pe baza scorului de relevanță textuală BM25.

10.2 Kibana

Platforma Kibana este utilizată ca instrument de dezvoltare și inspecție vizuală a datelor stocate în Elasticsearch, rulând local ca serviciu în Docker Compose. Prin intermediul consolei Dev Tools, Kibana a facilitat testarea, validarea și optimizarea mapping-urilor de index și a interogărilor JSON Query DSL înainte de transpunerea acestora în codul C#. Din motive de securitate, acest utilitar este folosit exclusiv în faza de dezvoltare și testare, nefiind expus public în mediul de producție.

10.3 Stack-ul Grafana: Prometheus și Loki

Observabilitatea sistemului este o cerință critică într-o arhitectură distribuită. Am construit un stack de monitorizare centralizat bazat pe Prometheus, Loki și Grafana.

Prometheus este utilizat pentru colectarea metricilor de performanță time-series [20]. În backend-ul .NET, am integrat pachetul `prometheus-net.AspNetCore`, care expune un endpoint securizat la adresa `/metrics` (memorie RAM, timp CPU, rate de request-uri și latențe). Serviciul de chat în Go își expune propriile metrice custom (conexiunile WebSocket active, latența de răspuns a modelului Ollama). Prometheus scanează aceste endpoint-uri la fiecare 15 secunde.

Loki funcționează ca un sistem de agregare a jurnalelor de execuție (*logs*). Containerele Docker trimit output-ul standard (`stdout`) către driverul Loki, care indexează doar label-urile metadata.

Grafana oferă interfața grafică de vizualizare, conectându-se la Prometheus și Loki

ca surse de date. Am construit un panou de control custom în Grafana organizat pe baza metodologiei RED, monitorizând rata interogărilor (Rate, request-uri/secundă), erorile (Errors, rata de eșec HTTP) și durata (Duration, latența p95 și p99). Datorită utilizării aceluiași identificator de trace (TraceId), operatorul poate corela un vârf de latență din Grafana cu log-urile specifice din Loki pentru diagnosticarea rapidă a blocajelor.

10.4 Kotlin — Serviciul de notificări push

Pentru a menține utilizatorii la curent cu stadiul licitațiilor și al comenzilor, am implementat un microserviciu dedicat trimerii de notificări push în browser, scris în Kotlin. Acesta rulează sub formă de Azure Functions (JVM runtime) [12]. Serviciul folosește două modalități de declanșare: **Timer Trigger** (scanează periodic baza de date PostgreSQL la fiecare 5 minute prin cron-ul `0 */5 * * * *` pentru a identifica licitațiile care se finalizează curând) și **HTTP Trigger** (endpoint apelat de backend-ul .NET la schimbarea statusului comenzilor).

Notificările sunt transmise utilizând protocolul standardizat **VAPID** [25]. Când un utilizator acceptă primirea notificărilor în frontend, browser-ul generează un obiect *Subscription* stocat în PostgreSQL în tabelul `push_subscriptions`. Serviciul Kotlin citește aceste date și trimite mesajele criptat către serverele push ale browserului utilizând biblioteca `nl.martijndwars:web-push`:

```
1 val pushService = PushService(vapidPublicKey, vapidPrivateKey, subject
   )
2 val notification = Notification(endpoint, userPublicKey, authAsBytes,
   payload.toByteArray())
3 pushService.send(notification)
```

Notificarea este recepționată de un Service Worker în React și afișată la nivel de sistem de operare chiar dacă tab-ul platformei este închis. Acest design decuplează trimerrea de mesaje de serverul web principal.

Capitolul 11

Aplicație administrativă (Next.js + Prisma)

Managementul operării unei platforme de e-commerce complexe necesită instrumente administrative avansate pentru moderarea utilizatorilor, monitorizarea comenzilor, validarea produselor din catalog și preluarea sesiunilor de suport escalated. În loc să integrăm aceste funcționalități sensibile în cadrul aplicației frontend destinate clienților (fapt ce ar fi mărit bundle-ul și ar fi expus cod sensibil public), am ales să dezvoltăm o aplicație administrativă independentă. Aceasta este construită utilizând framework-ul React Next.js 14 (App Router) și Prisma ORM, conectându-se direct la PostgreSQL cu drepturi depline. În acest capitol sunt prezentate detaliile de design, securitate, fluxurile de date și interfața de administrare a sistemului.

11.1 Motivația alegerii Next.js + Prisma

Next.js SSR previne expunerea logicii administrative în bundle-ul client; Prisma oferă type safety end-to-end din schema DB. Rutele administrative și componentele de vizualizare sunt randate în întregime pe server, iar browser-ul primește documentul HTML gata structurat, fără cod JavaScript de administrare atașat. Aplicația administrativă se conectează la PostgreSQL utilizând conexiunea directă asociată rolului `ks_admin`. Acest rol deține privilegii complete de citire și scriere pe toate tabelele bazei de date și ocolește în mod explicit politicile de securitate Row Level Security (BYPASS RLS), simplificând rapoartele din dashboard prin interogări directe.

11.2 Arhitectura aplicației administrative

Aplicația administrativă folosește structura modernă *App Router* introdusă în Next.js 14. Directorul principal al aplicației (`app/`) este organizat pe baza paginilor rulate implicit pe server (*React Server Components* — *RSC*).

Fluxul de acces este controlat prin fișierul de middleware (`middleware.ts`) aflat la rădăcina proiectului. Acesta interceptează toate cererile sosite pe rutele administrative protejate (sub calea `/app/*`). Middleware-ul extrage token-ul JWT din cookies-urile request-ului și îl validează server-side prin Firebase Admin SDK. Dacă token-ul lipsește, este expirat sau nu deține permisiunile de administrator, utilizatorul este redirecționat instantaneu către pagina de autentificare, blocând accesul înainte ca orice cod din pagină să fie executat pe server.

Instanțierea clientului Prisma se face prin intermediul unui pattern singleton definit în fișierul `lib/prisma.ts`, prevenind deschiderea de conexiuni multiple către PostgreSQL la fiecare reîncărcare la cald în timpul dezvoltării:

```
1 const globalForPrisma = global as unknown as { prisma: PrismaClient };
2 export const prisma = globalForPrisma.prisma || new PrismaClient();
3 if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma =
  prisma;
```

Rutele de API native (`app/api/*`) sunt utilizate exclusiv pentru gestionarea operațiunilor mutative trimise prin formulare și pentru a acționa ca un proxy securizat către serviciile backend externe.

11.3 Funcționalități administrative principale

Tabloul de bord principal (Dashboard) oferă administratorilor o imagine de ansamblu asupra performanței platformei în timp real, calculând pe server numărul de comenzi și veniturile prin Prisma pentru a asigura încărcări rapide sub 100ms.

Secțiunea de **Management Catalog (CRUD)** permite vizualizarea și paginarea produselor, stocurilor și mărimilor active, precum și adăugarea de elemente noi, sincronizate asincron cu indexul Elasticsearch prin apeluri către backend-ul .NET.

Panoul de **Gestiune Utilizatori** facilitează listarea conturilor înregistrate, blocarea utilizatorilor abuzivi (`is_banned = true`) și validarea cererilor de seller prin actualizarea tabelelor locale și a claims-urilor Firebase.

Secțiunea de **Sesiuni Escalated Chat** preia din PostgreSQL sesiunile marcate cu flag-ul de escaladare de către LLM din Go. Operatorul poate răspunde direct în chat-ul preluat prin intermediul unui API proxiat din Next.js către WebSocket-ul din Go.

11.4 Deployment și izolare

Imaginea Docker pentru aplicația Next.js este construită utilizând un proces optimizat multi-stage bazat pe modul standalone oferit nativ de framework:

```
1 # Stage 1: Build source
2 FROM node:20-alpine AS builder
3 WORKDIR /app
4 COPY package*.json ./
5 RUN npm ci
6 COPY . .
7 RUN npx prisma generate
8 RUN npm run build
9
10 # Stage 2: Final minimal image
11 FROM node:20-alpine AS runner
12 WORKDIR /app
13 COPY --from=builder /app/.next/standalone ./
14 COPY --from=builder /app/.next/static ./next/static
15 COPY --from=builder /app/public ./public
16 EXPOSE 3001
17 CMD ["node", "server.js"]
```

Utilizarea modului standalone compilează doar fișierele strict necesare rulării în producție, reducând dimensiunea imaginii Docker finale la mai puțin de 120 MB.

Variabila de mediu `DATABASE_URL` stocată securizat în configurarea containerului indică stringul de conectare cu drepturi administrative `ks_admin`. La fiecare pornire a containerului, scriptul de entrypoint execută automat comanda `npx prisma migrate deploy`, asigurând că migrările schemei de bază de date sunt aplicate înainte de pornirea serverului.

Aplicația administrativă rulează izolată pe portul 3001, complet independentă de

frontend-ul utilizatorilor finali (portul 3000). Această separare fizică și de domenii prezintă o barieră importantă împotriva atacurilor XSS în aplicația publică, consolidând arhitectura multi-strat (*defense in depth*) a platformei KickSneak.

Capitolul 12

Testare

Asigurarea funcționării corecte și a stabilității unei platforme distribuite poliglote necesită o strategie de validare riguroasă, aplicată la niveluri diferite de granularitate. Pentru platforma KickSneak, am proiectat și implementat un plan complet de testare, acoperind teste unitare de logică internă, teste de integrare a endpoint-urilor HTTP și scenarii automate de la un capăt la altul (E2E) pe interfața grafică. În acest capitol sunt descrise arhitectura suitei de teste, utilizarea bibliotecilor de mock-uire, rularea containerelor ephemere pentru baze de date de test și integrarea testelor în fluxul de integrare continuă (CI/CD).

12.1 Strategia de testare

Procesul de asigurare a calității codului în platforma KickSneak respectă structura clasică a piramidei de testare propusă de Martin Fowler [6]. Aceasta organizează testele pe trei niveluri: **Unit Tests** (teste numeroase și rapide care izolează logica de business prin mock-uri), **Integration Tests** (teste de colaborare între componente și baza de date persistentă) și **End-to-End Tests** (teste de automatizare a browser-ului pe interfața React reală). Cerința academică minimă a tezei de a include cel puțin 10 teste unitare funcționale a fost acoperită și depășită pe deplin, asamblând o acoperire de cod robustă pentru cazurile critice.

12.2 Unit tests — xUnit + NSubstitute + Shouldly

În backend-ul .NET, testele unitare sunt scrise utilizând framework-ul xUnit [26]. Pentru crearea rapidă a obiectelor mock am ales biblioteca NSubstitute [15], iar pentru scrierea aserțiunilor clare și lizibile am utilizat Shouldly. Fiecare test respectă structura

standard AAA (*Arrange / Act / Assert*).

Am dezvoltat suite de teste detaliate pentru serviciile cheie ale platformei:

AuctionService:

- ▷ `PlaceBid_HappyPath`: Validează plasarea unui bid valid, scrierea în Redis/PostgreSQL și transmiterea prin SignalR.
- ▷ `PlaceBid_BelowCurrentPrice`: Verifică aruncarea excepției `BusinessException` la introducerea unei sume inferioare ofertei curente.
- ▷ `PlaceBid_AutoBidChain`: Testează recursivitatea auto-bid-ului, limitată de guardul `MaxAutoBidDepth = 10`.

OrderService:

- ▷ `CreateOrder_HappyPath`: Validează crearea unei comenzi valide, decrementarea stocului și generarea sesiunii Stripe.
- ▷ `CreateOrder_OutOfStock`: Verifică blocarea tranzacției și aruncarea excepției când cantitatea din stoc este zero.

În mod similar, microserviciul Go Chat folosește pachetul nativ `testing` pentru validarea logicii de asistență virtuală. Testul `TestSendMessage_EscalateIntent` trimite un mesaj cu intenție de plângere, validând că serviciul actualizează statusul sesiunii în PostgreSQL ca fiind "escalated" și returnează payload-ul corespunzător.

12.3 Integration tests — ASP.NET Core TestServer

Testele de integrare din backend-ul .NET utilizează clasa `WebApplicationFactory<Program>` pentru a porni o instanță în memorie a întregului server web ASP.NET Core. Rutele HTTP sunt interogate utilizând un client HTTP de test configurat automat.

Pentru a asigura o validare fidelă fără a polua baza de date de dezvoltare, am implementat două strategii pentru persistență:

- ▷ **InMemory Database Provider**: Pentru testarea rapidă a endpoint-urilor simple, baza relațională este înlocuită cu un provider in-memory local.

- ▷ **Testcontainers:** Pentru testarea politicilor RLS și a instrucțiunilor de switch de rol, biblioteca Testcontainers [1] pornește asincron un container Docker PostgreSQL efemer înaintea rularii suitei și îl distruge la final.

Testele de integrare validează rutele critice:

- ▷ `POST /api/v1/auctions/{id}/bid`: Trimite o cerere autenticată cu un token JWT de test, verificând returnarea statusului 200 OK și actualizarea prețului.
- ▷ `GET /api/v1/products/search`: Validează că interogările transmise către instanța Elasticsearch mockuită întorc rezultate structurate corespunzător.

12.4 Automatizare și CI/CD

Testele de tip End-to-End validează interfața grafică React utilizând biblioteca Playwright [13], rulând o instanță headless de browser Chromium pe codul local. S-a dezvoltat un scenariu complet care validează fluxul principal de cumpărare:

1. Căutarea produsului: Introduce textul în bara de căutare din interfață și validează sugestiile live.
2. Adăugarea în coș: Selectează produsul, se autentifică prin contul Firebase de test și adaugă articolul în coșul virtual.
3. Finalizarea tranzacției Stripe: Completează adresa, trece prin Stripe Checkout introducând cardul de test și validează ecranul de confirmare.

În caz de eșec, Playwright capturează ecranul (`page.screenshot()`) pentru diagnosticare vizuală rapidă.

Pentru a asigura calitatea codului la fiecare push sau pull request, am integrat suita de teste într-un pipeline de integrare continuă în GitHub Actions (`test.yml`). Pipeline-ul rulează în paralel:

- ▷ `unit-tests`: Instalează .NET 10 și rulează comanda:

```
1 dotnet test --configuration Release --collect:"XPlat Code  
2 Coverage"
```

- ▷ `go-tests`: Instalează Go și execută testele unitare pentru serviciul de chat.

- ▷ `integration-tests`: Pornește PostgreSQL în runner, rulează migrările și execută testele de integrare.

La finalizare, statusul pipeline-ului este actualizat direct în repository, prevenind regresiunile în ramurile principale.

Capitolul 13

Concluzii și direcții viitoare

Dezvoltarea platformei KickSneak a reprezentat o oportunitate deosebită de a explora integrarea practică a unor paradigme și tehnologii software moderne în cadrul unui sistem distribuit complex. Pe parcursul proiectării și implementării, am abordat o gamă largă de provocări tehnice reale — de la concurența ridicată pe tranzacții în timp real și securizarea granulară la nivel de date, până la antrenarea de rețele neuronale pentru recomandări și implementarea asistenței conversaționale cu modele mari de limbaj. În acest capitol final sunt prezentate concluziile generale ale lucrării, performanțele și rezultatele obținute, limitările asumate ale implementării curente și direcțiile viitoare de dezvoltare ale platformei.

13.1 Rezultate obținute

Rezultatul principal al acestei lucrări este o platformă marketplace de e-commerce complet funcțională, care integrează cu succes un număr semnificativ de module interconectate. Întregul sistem distribuit, compus din 8 servicii containerizate distincte (frontend React, admin Next.js, backend .NET, chat Go, recommender Python, baze de date PostgreSQL, cache Redis și index search Elasticsearch), poate fi pornit local printr-o singură comandă:

```
1 docker compose up --build
```

Pe parcursul dezvoltării, am validat mai multe decizii arhitecturale critice:

- ▷ **Arhitectura poliglotă:** Am demonstrat eficiența utilizării a 6 limbaje de programare distincte (C#, TypeScript, Go, Python, Java și Kotlin), fiecare fiind ales în mod justificat pentru caracteristicile sale de performanță (Go pentru conexiuni WebSocket concurente, Python pentru logica matematică a rețelei neuronale, C# ca nucleu tranzacțional stabil și Next.js/React pentru interfețe).

- ▷ **Securitate robustă la nivel de baze de date:** Am implementat un model hibrid format din Firebase Authentication corelat cu un sistem RBAC cu 6 roluri PostgreSQL și politici de Row Level Security (RLS) la nivel de tabel, asigurate prin switch dinamic de conexiune cu instrucțiuni SET LOCAL ROLE. Această structură oferă o protecție excelentă a datelor sensibile ale utilizatorilor.
- ▷ **Performanță în timp real:** Sistemul de licitații implementează un pattern de tip *Redis-first* cu control optimist al concurenței, garantând latențe de bidding sub-milisecundă și eliminând blocajele de scriere concurentă din PostgreSQL.
- ▷ **Decuplare asincronă:** Utilizarea primitivelor de tip `Channel<T>` în ASP.NET Core a facilitat implementarea de servicii background persistente pentru sincronizarea indexului Elasticsearch și expedierea de notificări, decuplând complet timpul de răspuns HTTP de operațiunile lente de I/O de fundal.

Lucrarea acoperă pe deplin cerințele academice stabilite pentru o lucrare de absolvire, oferind în același timp o structură de cod conformă cu standardele din industria software contemporană, pregătită pentru scalare.

13.2 Limitări și provocări

În ciuda rezultatelor pozitive, implementarea curentă prezintă o serie de limitări din punct de vedere tehnic și operațional, care trebuie asumate cu onestitate academică:

- ▷ **Date de antrenament sintetice:** Rețeaua neuronală NCF antrenată în PyTorch folosește date generate artificial prin scriptul de seed semi-sintetic. Metricile Hit Rate și NDCG obținute sunt ilustrative pentru funcționarea algoritmului, dar antrenarea pe un set de date real de comportament al utilizatorilor ar putea releva probleme de personalizare sau necesitatea ajustării hiperparametrilor.
- ▷ **Latența LLM-ului local:** Rularea modelului *llama3.1:8b* prin Ollama direct pe mașina de dezvoltare locală generează o latență de răspuns sesizabilă (timpul până la primul token fiind între 2 și 5 secunde, în funcție de încărcarea GPU-ului). În mediul de producție, acest comportament ar necesita servicii cloud dedicate sau API-uri externe comerciale.
- ▷ **Bypass RLS la citiri în .NET:** Din considerente de performanță a query-urilor complexe pe catalog (unde join-urile multiple pe tabele mari ar fi fost grav încetinite de politicile RLS), rolul `ks_owner` utilizat de Entity Framework Core ocolește RLS-ul

la operațiunile de citire (SELECT), securitatea RLS fiind aplicată strict la scrieri. O securitate completă ar fi presupus activarea regulii `FORCE ROW LEVEL SECURITY`, cu prețul unei latențe sporite.

- ▷ **Deployment exclusiv local:** Deși aplicațiile sunt containerizate prin Dockerfile-uri multi-stage, platforma rulează doar pe o singură mașină fizică în mediul local. Absența unui deployment activ în cloud (pe servicii comerciale precum Azure sau AWS) constituie o limitare operațională curentă.

13.3 Direcții viitoare

Pentru a aduce platforma KickSneak la stadiul de produs de producție real, am stabilit următoarele direcții viitoare de dezvoltare:

- ▷ **Migrarea completă în Cloud public:** Propunem deployment-ul containerelor în cloud utilizând servicii gestionate, cum ar fi Azure Container Apps sau Azure Kubernetes Service (AKS). Baza de date ar fi migrată către Azure Database for PostgreSQL, cache-ul către Azure Cache for Redis, iar stocarea Azurite ar fi înlocuită de un Azure Storage Account real.
- ▷ **Dezvoltarea de aplicații mobile:** Având în vedere că toate endpoint-urile din backend sunt proiectate sub formă de servicii web REST API, o direcție prioritară este construirea de aplicații mobile pentru Android și iOS utilizând framework-uri hibride precum React Native sau Flutter, reutilizând integral logica de business existentă.
- ▷ **Îmbunătățirea algoritmilor AI:** Rețeaua NCF poate fi extinsă pentru a include caracteristici suplimentare (*features*), cum ar fi brandul preferat, mărimea utilizatorului sau interogările de search recente. De asemenea, se dorește implementarea unui pipeline de reantrenare online a modelului, care să actualizeze embeddings-urile în mod regulat pe baza noilor interacțiuni din baza de date.
- ▷ **Securitate avansată și conformitate:** Integrarea de mecanisme de autentificare biometrică (*Passkeys*) și autentificare cu doi factori (MFA) native din Firebase, precum și auditarea completă a plăților Stripe pentru obținerea certificării de securitate PCI-DSS în faza de producție.

În concluzie, proiectul a atins obiectivele stabilite și oferă o bază solidă, scalabilă și modulară, reprezentând o demonstrație practică reușită a modului în care pot fi integrate bunele practici tehnice contemporane pentru a construi un sistem software distribuit modern.

Bibliografie

- [1] AtomicJar, Inc. Testcontainers: Throwaway instances of databases, message brokers, web browsers, 2026.
- [2] Katherine Cox-Buday. *Concurrency in Go: Tools and Techniques for Developers*. O'Reilly Media, 2017.
- [3] Docker, Inc. Docker compose overview and multi-container orchestration, 2026.
- [4] Elastic Search. Elasticsearch guide reference (bm25 relevance scoring), 2026.
- [5] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional, 2003.
- [6] Martin Fowler. The practical test pyramid, 2012.
- [7] Brad Frost. *Atomic Design*. Brad Frost, 2016.
- [8] Google LLC. Firebase authentication documentation, 2026.
- [9] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering. In *Proceedings of the 26th International Conference on World Wide Web*, pages 173–182, 2017.
- [10] Yifan Hu, Yehuda Koren, and Chris Volinsky. Collaborative filtering for implicit feedback datasets. *Proceedings of the IEEE International Conference on Data Mining*, pages 263–272, 2008.
- [11] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
- [12] Microsoft Corporation. Azure functions kotlin developer guide, 2026.
- [13] Microsoft Corporation. Playwright: Fast and reliable end-to-end testing for modern web apps, 2026.

- [14] Microsoft Corporation. System.threading.channels library overview, 2026.
- [15] NSubstitute. Nsubstitute: A friendly substitute for .net mocking libraries, 2026.
- [16] Ollama. Ollama: Run llama 3.1 and qwen models locally, 2026.
- [17] PCI Security Standards Council. Payment card industry data security standard (pci dss) requirements and security assessment procedures, 2024.
- [18] Pmndrs. Zustand: Bearish state management library for react, 2026.
- [19] PostgreSQL Global Development Group. Postgresql 16 documentation: Row level security policies, 2026.
- [20] Prometheus Authors. Prometheus: An open-source monitoring system with a dimensional data model, 2026.
- [21] Quartz.NET. Quartz.net documentation: Job scheduling and execution, 2026.
- [22] Redis Ltd. Redis transactions: Watch, multi, exec, 2026.
- [23] Guy Shani and Asela Gunawardana. Evaluating recommendation systems. In *Recommender Systems Handbook*, pages 257–297. Springer, 2011.
- [24] Stripe, Inc. Stripe api reference and webhook integration, 2026.
- [25] Martin Thomson and Peter Beverloo. Voluntary application server identification (vapid) for web push. RFC 8292, IETF, 2017.
- [26] xUnit.net. xunit.net: Unit testing framework for .net, 2026.